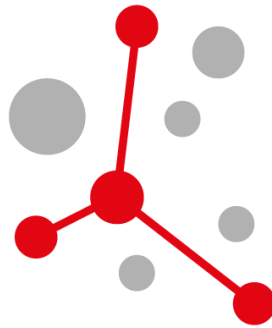


CONNECTING THE DOTS

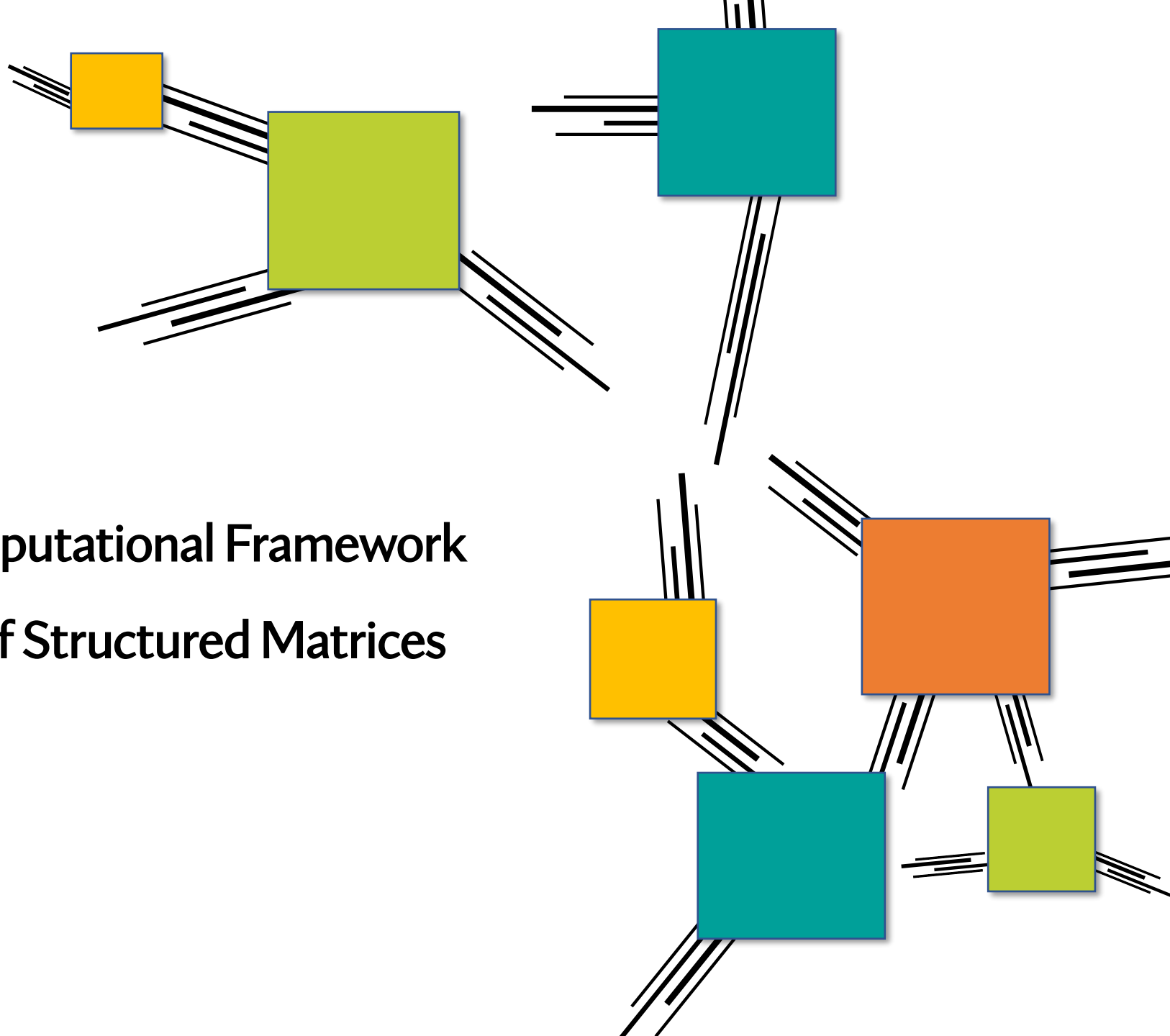


ISC

High Performance

JUNE 10 – 13, 2025 | HAMBURG, GERMANY

sTiles: An Accelerated Computational Framework for Sparse Factorizations of Structured Matrices

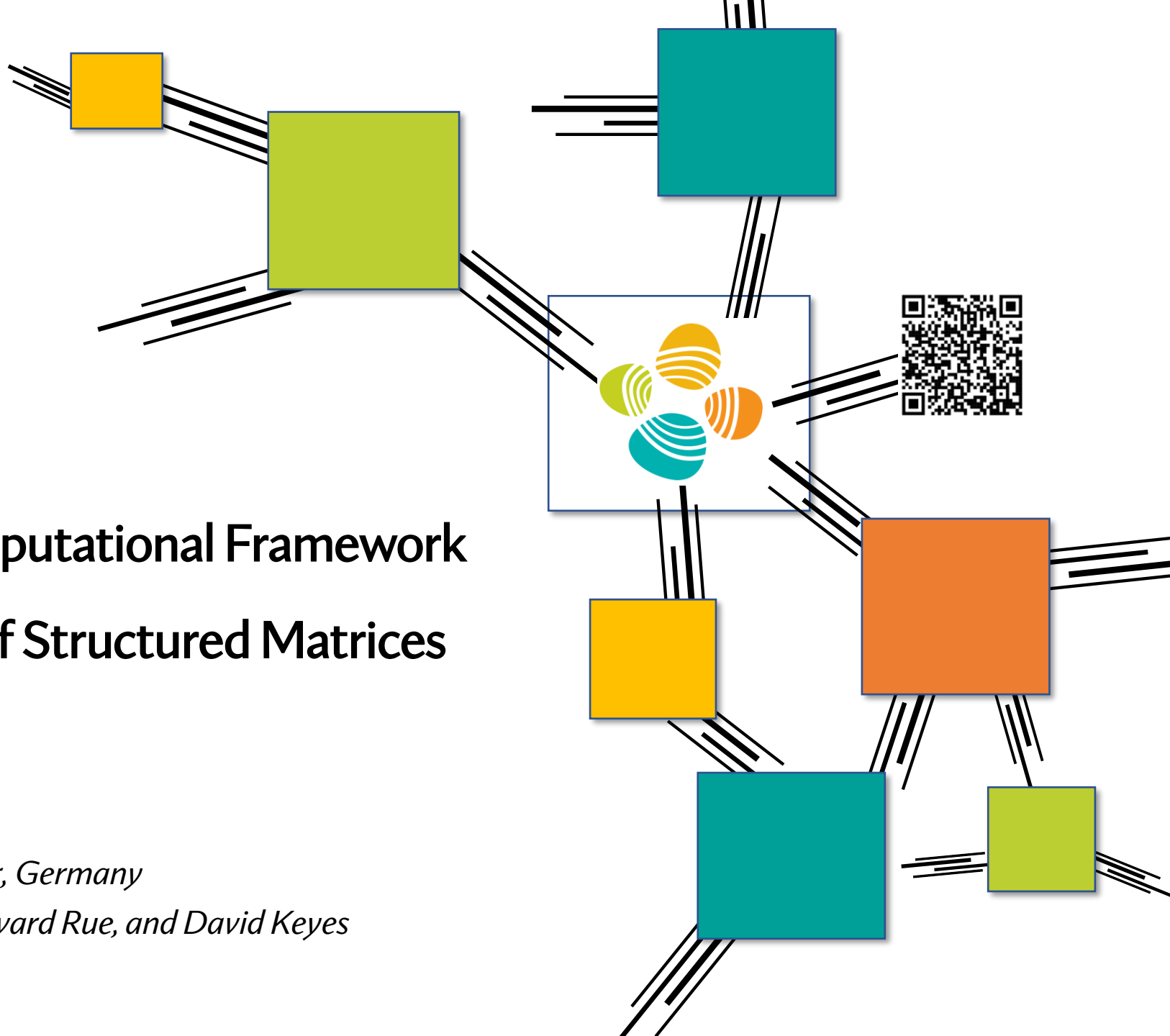


sTiles: An Accelerated Computational Framework for Sparse Factorizations of Structured Matrices

Esmail Abdul Fattah, KAUST

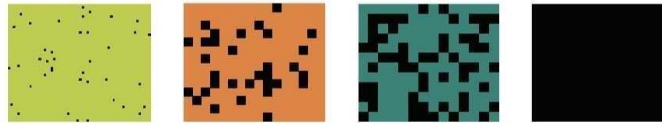
ISC High Performance 2025, Hamburg, Germany

In collaboration with Hatem Ltaief, Håvard Rue, and David Keyes



In this talk we introduce **sTiles**: a tile-based solver framework designed for structured symmetric sparse matrices.

- Combines the strengths of sparse and dense solvers:
 - Exploits sparsity to reduce memory and computation.
 - Uses dense tiles to unlock high-performance compute kernels and parallelism.
- Tailored for modern HPC systems:
 - Static scheduling for shared memory.
 - GPU acceleration with CUDA streams.





Where Do These Structured Matrices Come From?



INLA, a fast Bayesian inference framework used in real scientific applications - but increasingly limited by solver performance.





Where Do These Structured Matrices Come From?



INLA, a fast Bayesian inference framework used in real scientific applications - but increasingly limited by solver performance.

- These structured matrices appear in:
 - Spatial and spatiotemporal models in epidemiology, geostatistics, environmental sciences.





Where Do These Structured Matrices Come From?



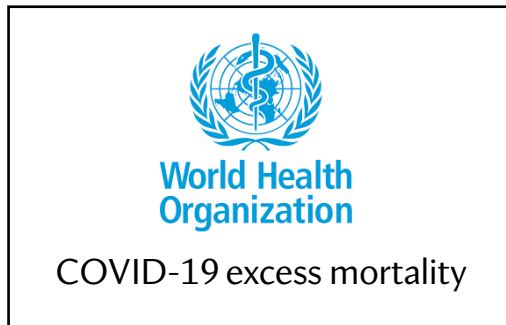
INLA, a fast Bayesian inference framework used in real scientific applications - but increasingly limited by solver performance.

- These structured matrices appear in:
 - Spatial and spatiotemporal models in epidemiology, geostatistics, environmental sciences.
 - Bayesian disease mapping and public health models:

Where Do These Structured Matrices Come From?

INLA, a fast Bayesian inference framework used in real scientific applications - but increasingly limited by solver performance.

- These structured matrices appear in:
 - Spatial and spatiotemporal models in epidemiology, geostatistics, environmental sciences.
 - Bayesian disease mapping and public health models:



Where Do These Structured Matrices Come From?

INLA, a fast Bayesian inference framework used in real scientific applications - but increasingly limited by solver performance.

■ These structured matrices appear in:

- Spatial and spatiotemporal models in epidemiology, geostatistics, environmental sciences.
- Bayesian disease mapping and public health models:



World Health
Organization

COVID-19 excess mortality



CENTERS FOR DISEASE
CONTROL AND PREVENTION

Teen birth and drug mortality rates

Where Do These Structured Matrices Come From?

INLA, a fast Bayesian inference framework used in real scientific applications - but increasingly limited by solver performance.

■ These structured matrices appear in:

- Spatial and spatiotemporal models in epidemiology, geostatistics, environmental sciences.
- Bayesian disease mapping and public health models:



World Health
Organization

COVID-19 excess mortality



CENTERS FOR DISEASE
CONTROL AND PREVENTION

Teen birth and drug mortality rates



UNITED NATIONS

Global population forecasting

Where Do These Structured Matrices Come From?

INLA, a fast Bayesian inference framework used in real scientific applications - but increasingly limited by solver performance.

■ These structured matrices appear in:

- Spatial and spatiotemporal models in epidemiology, geostatistics, environmental sciences.
- Bayesian disease mapping and public health models:



World Health
Organization

COVID-19 excess mortality



CENTERS FOR DISEASE
CONTROL AND PREVENTION

Teen birth and drug mortality rates



UNITED NATIONS

Global population forecasting

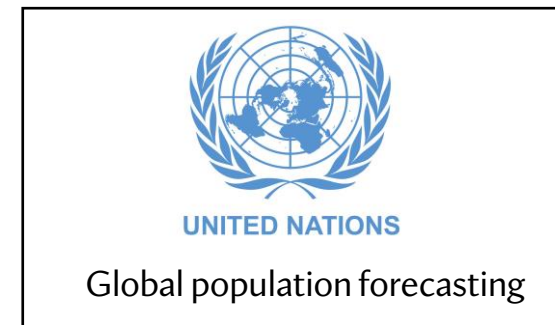
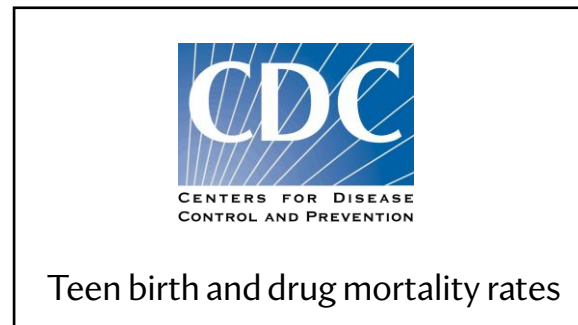
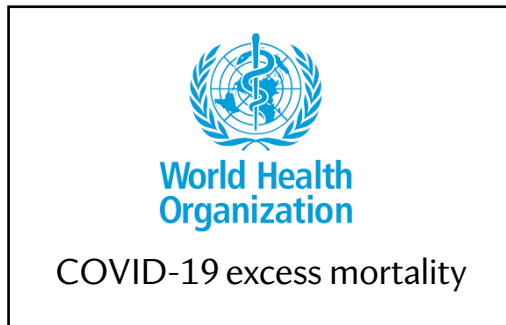
- Climate and geoscience applications using the Stochastic Partial Differential Equations (SPDE) approach.

Where Do These Structured Matrices Come From?

INLA, a fast Bayesian inference framework used in real scientific applications - but increasingly limited by solver performance.

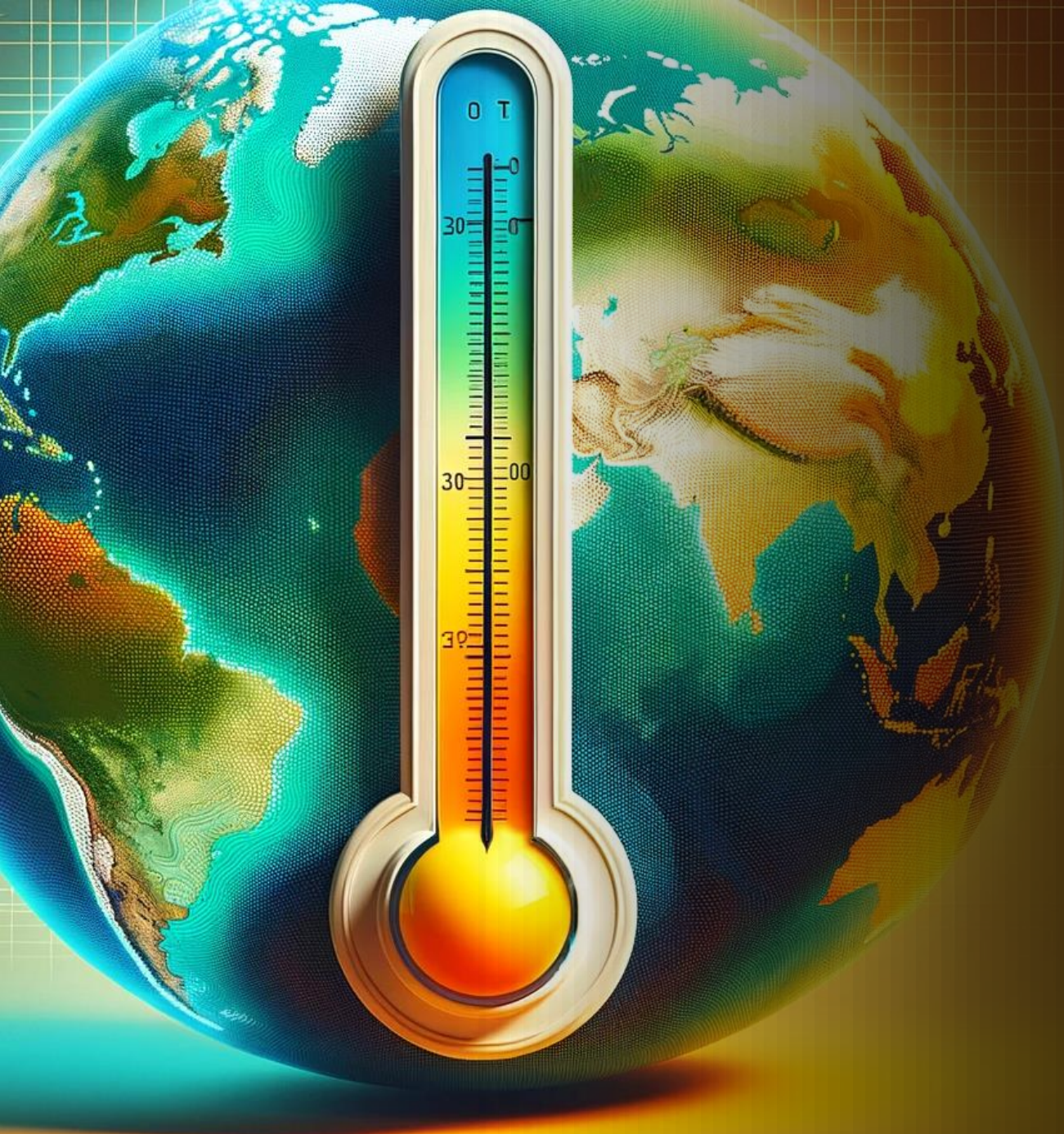
- These structured matrices appear in:

- Spatial and spatiotemporal models in epidemiology, geostatistics, environmental sciences.
- Bayesian disease mapping and public health models:



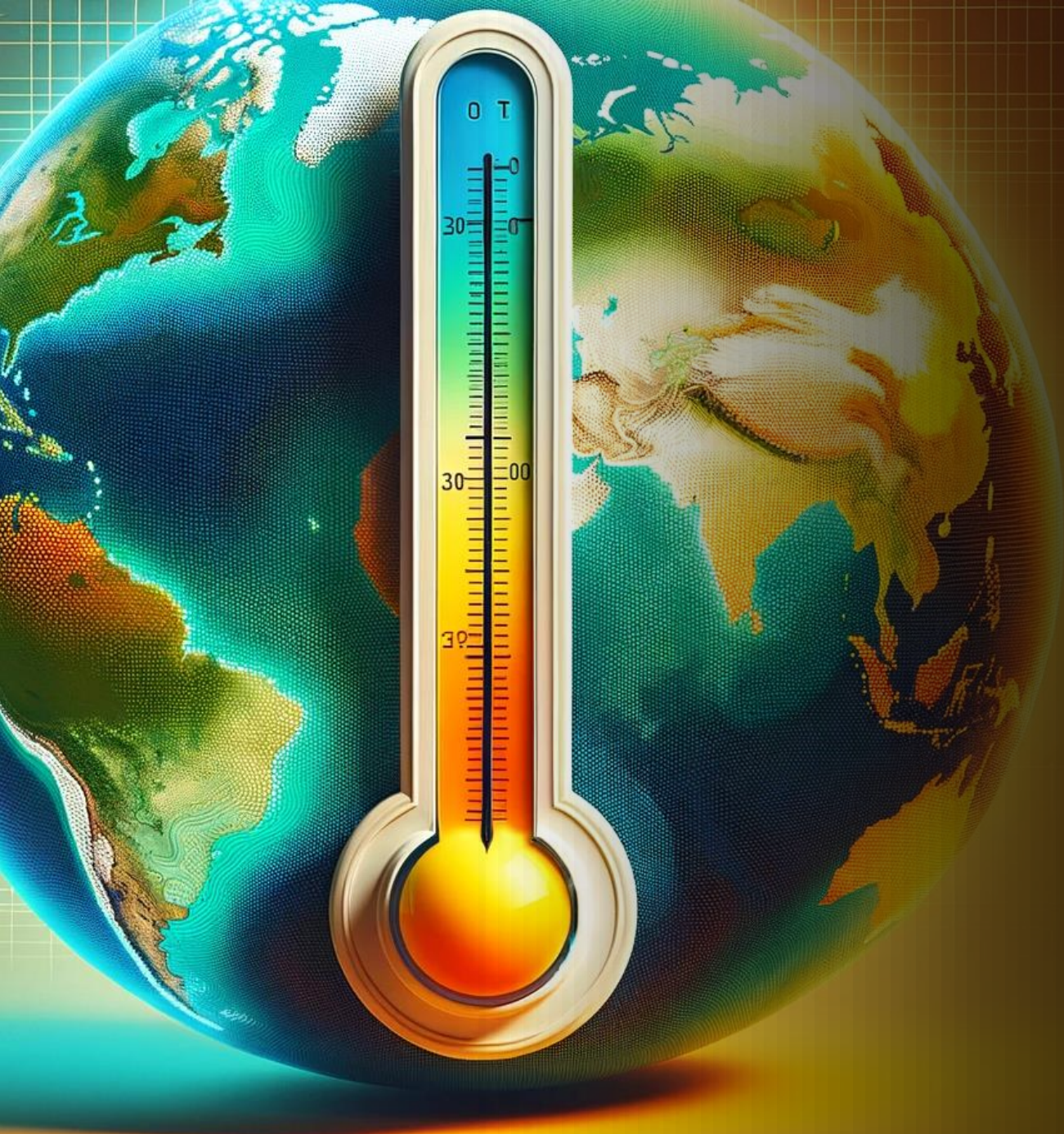
- Climate and geoscience applications using the Stochastic Partial Differential Equations (SPDE) approach.

- Models with large latent fields and many fixed effects → arrowhead precision matrices (inverse of covariance) .



Modeling Global Temperature Patterns using INLA

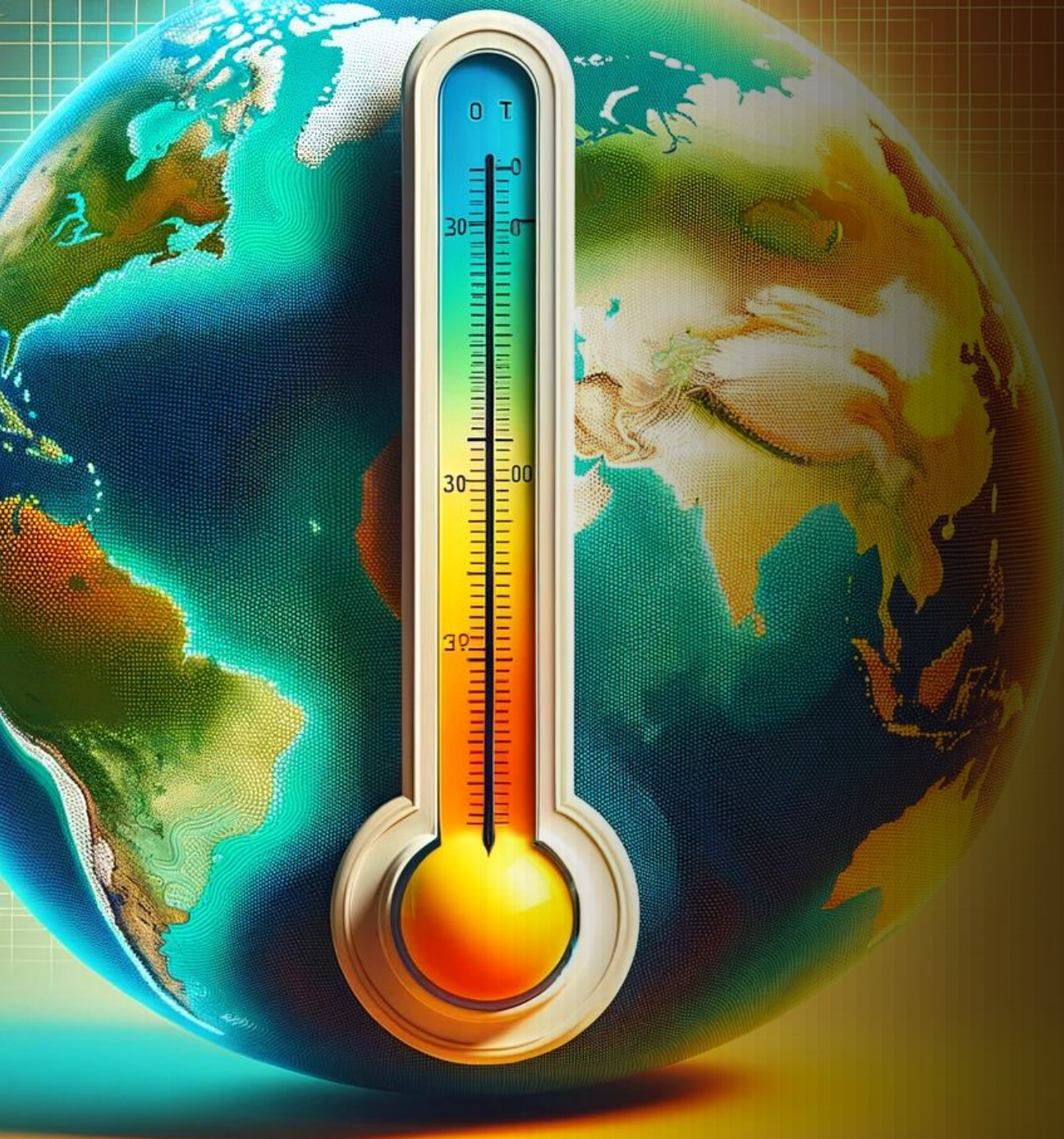




Modeling Global Temperature Patterns using INLA

Why INLA?



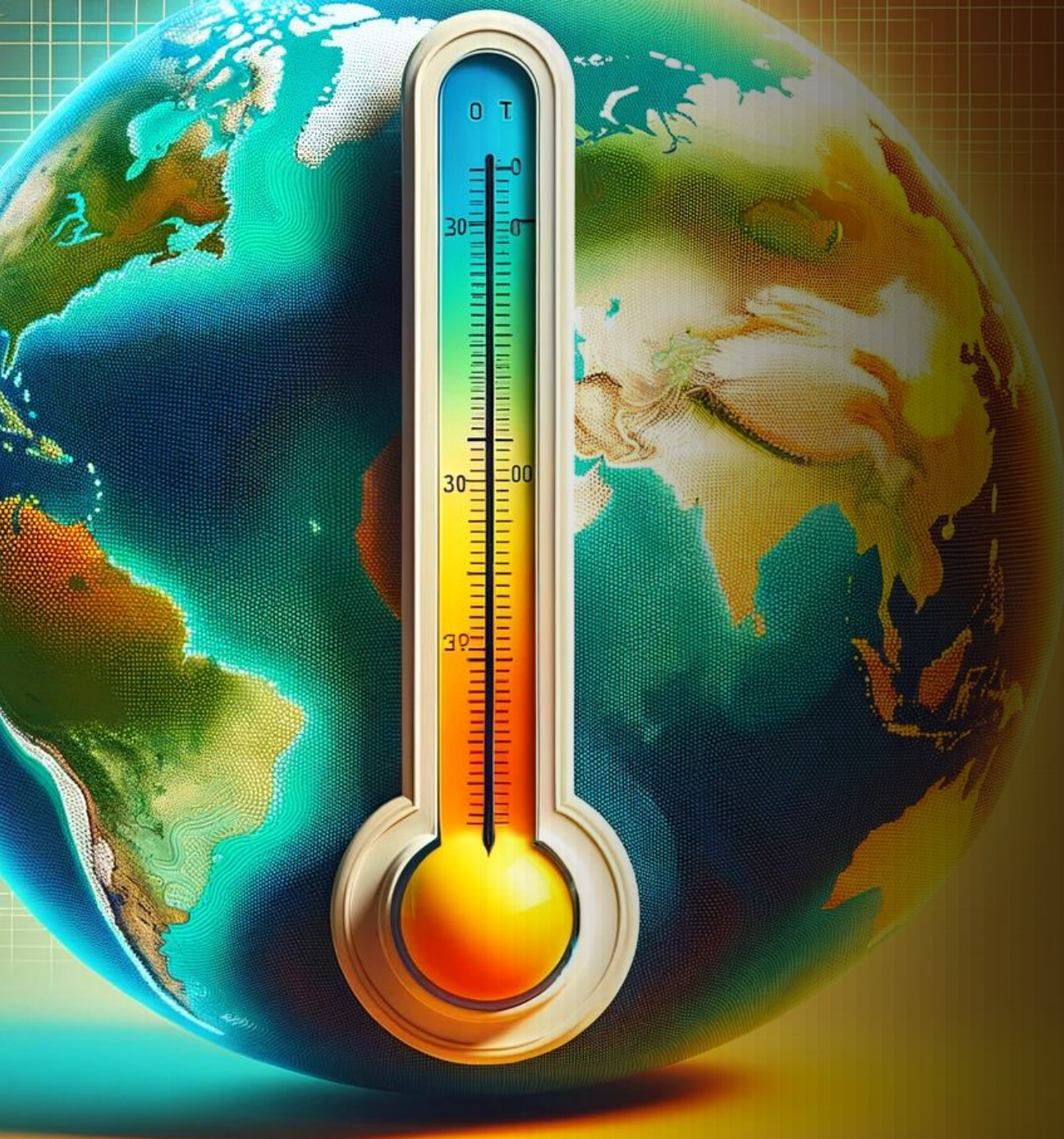


Modeling Global Temperature Patterns using INLA

Why INLA?

- Traditional Markov Chain Monte Carlo methods struggle with scalability.



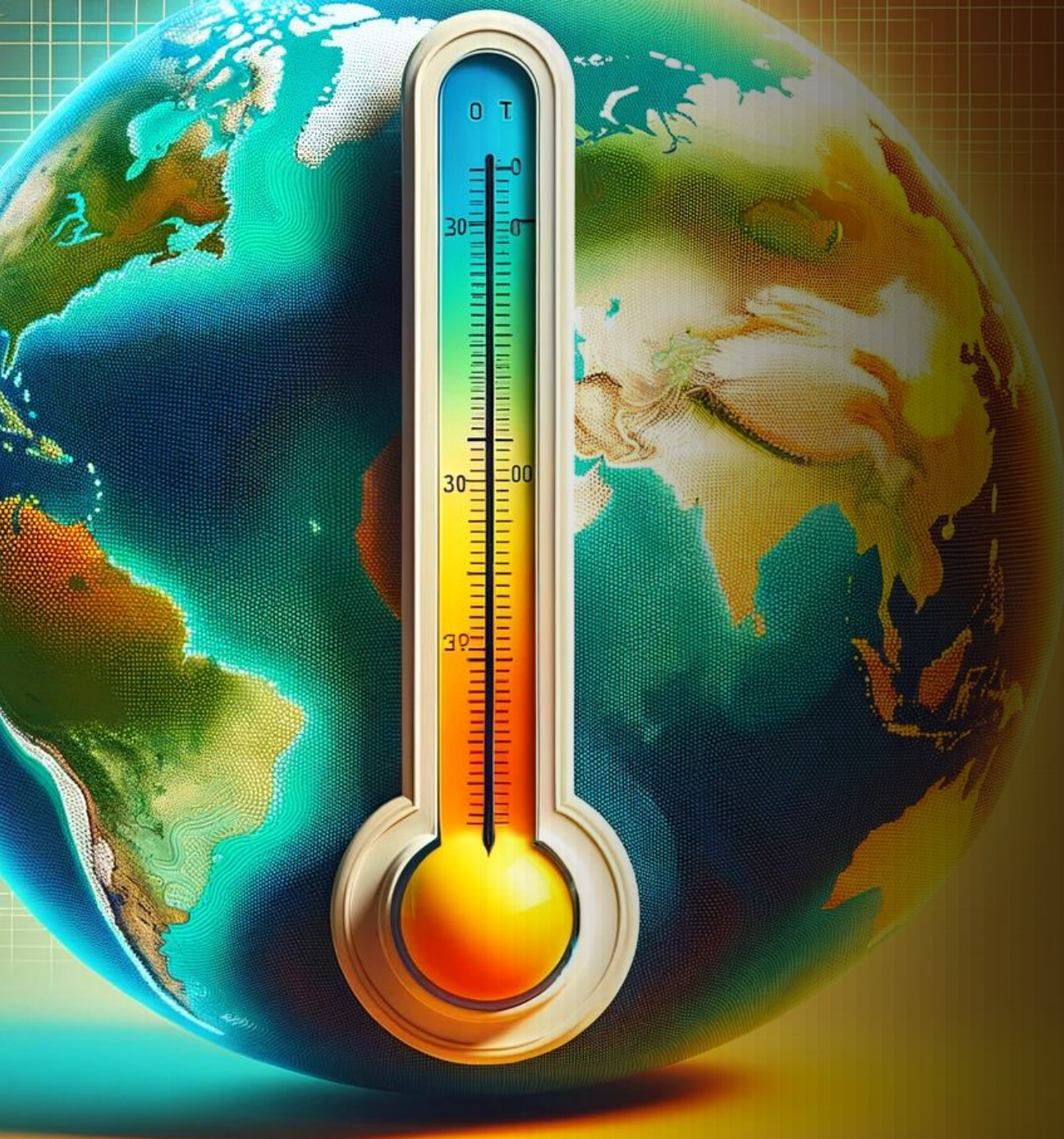


Modeling Global Temperature Patterns using INLA

Why INLA?

- Traditional Markov Chain Monte Carlo methods struggle with scalability.
- INLA offers fast, accurate, and deterministic inference for Latent Gaussian Models.



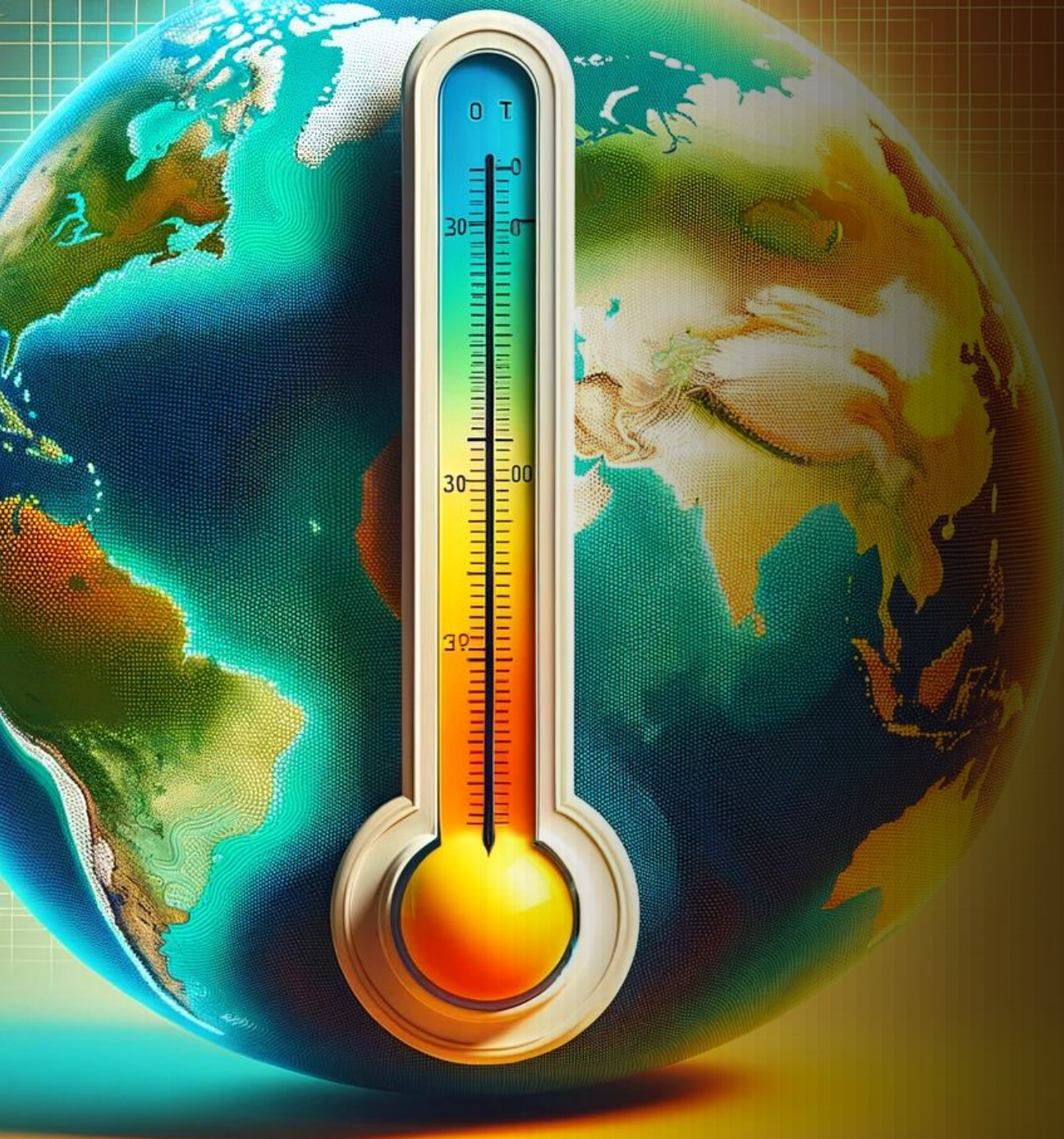


Modeling Global Temperature Patterns using INLA

Real-World Application

- Modeling global temperature patterns using data from over 13,000 weather stations.



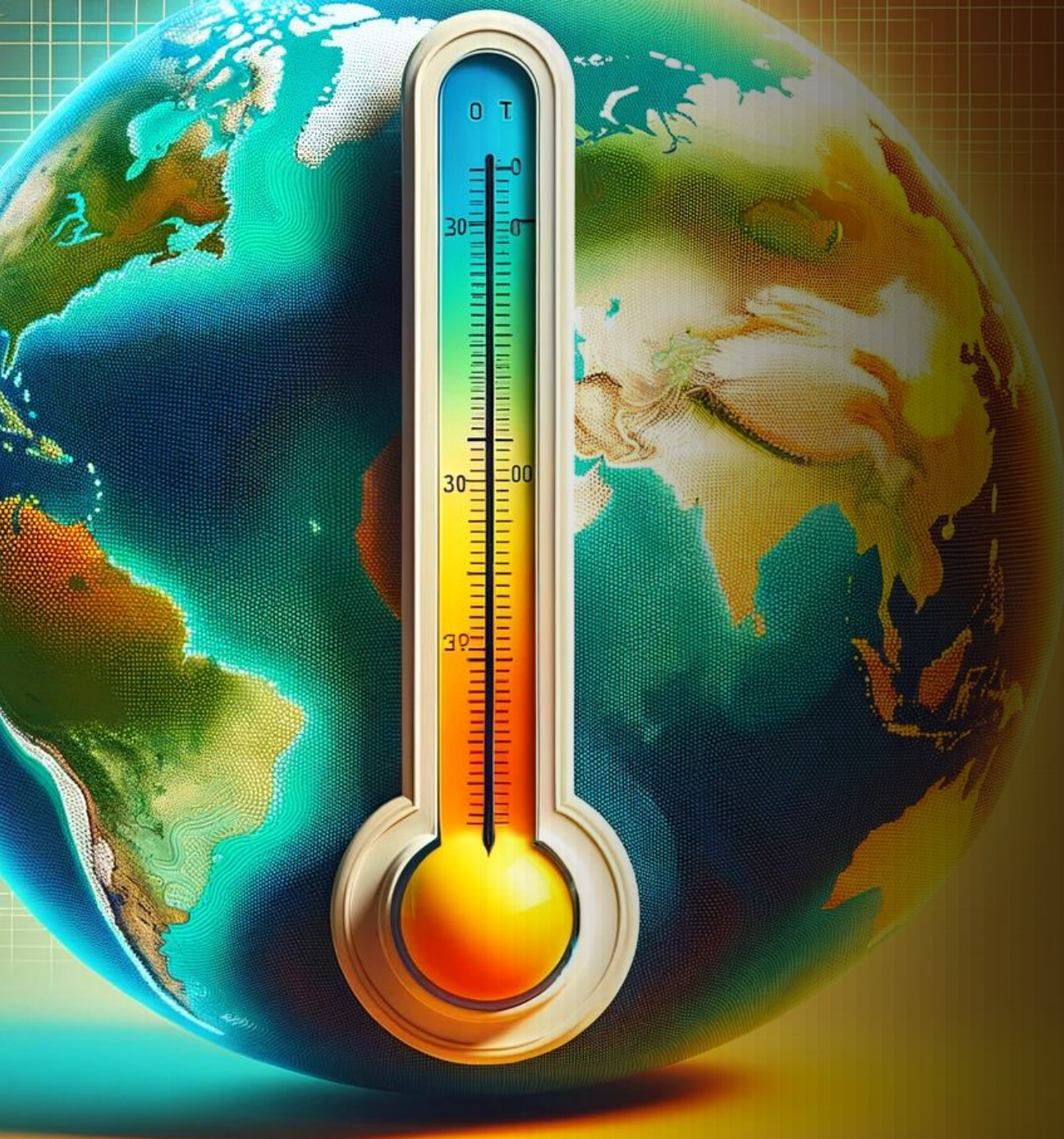


Modeling Global Temperature Patterns using INLA

Real-World Application

- Modeling global temperature patterns using data from over 13,000 weather stations.
- Daily high/low temps for all of 2022 → nearly 5 million data records.



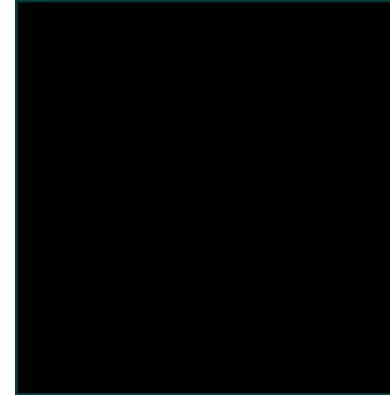
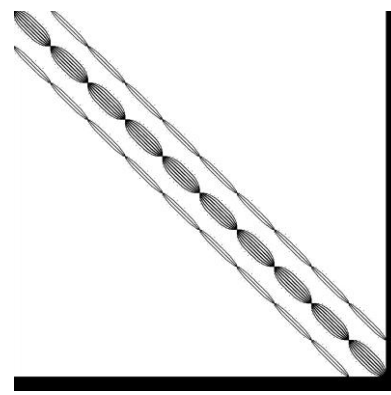
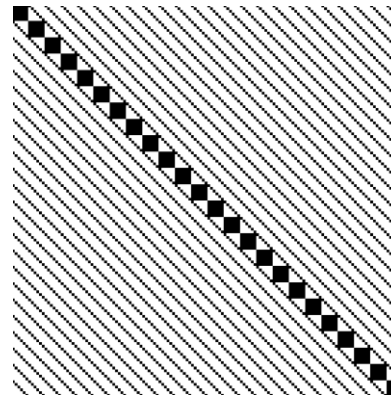
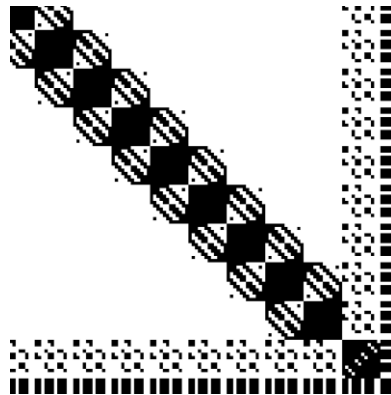
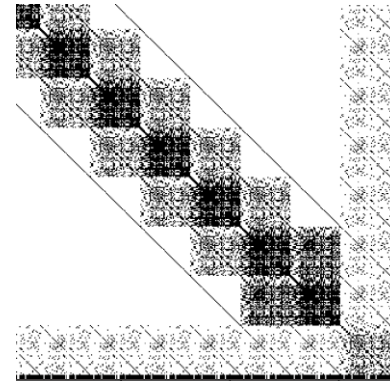
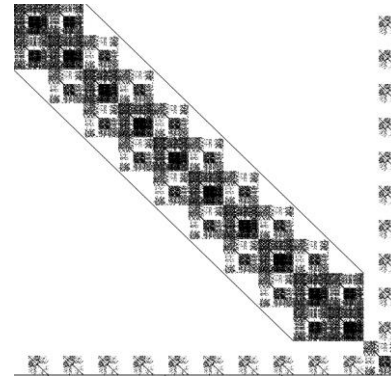
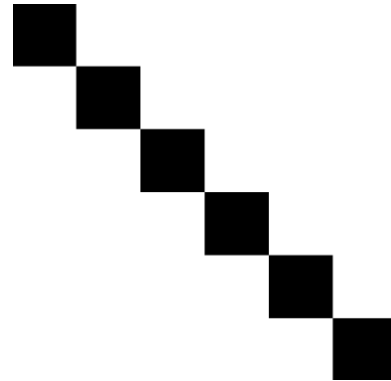
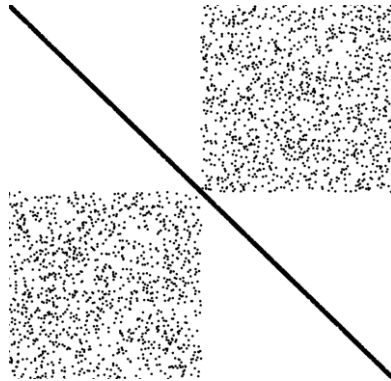


Modeling Global Temperature Patterns using INLA

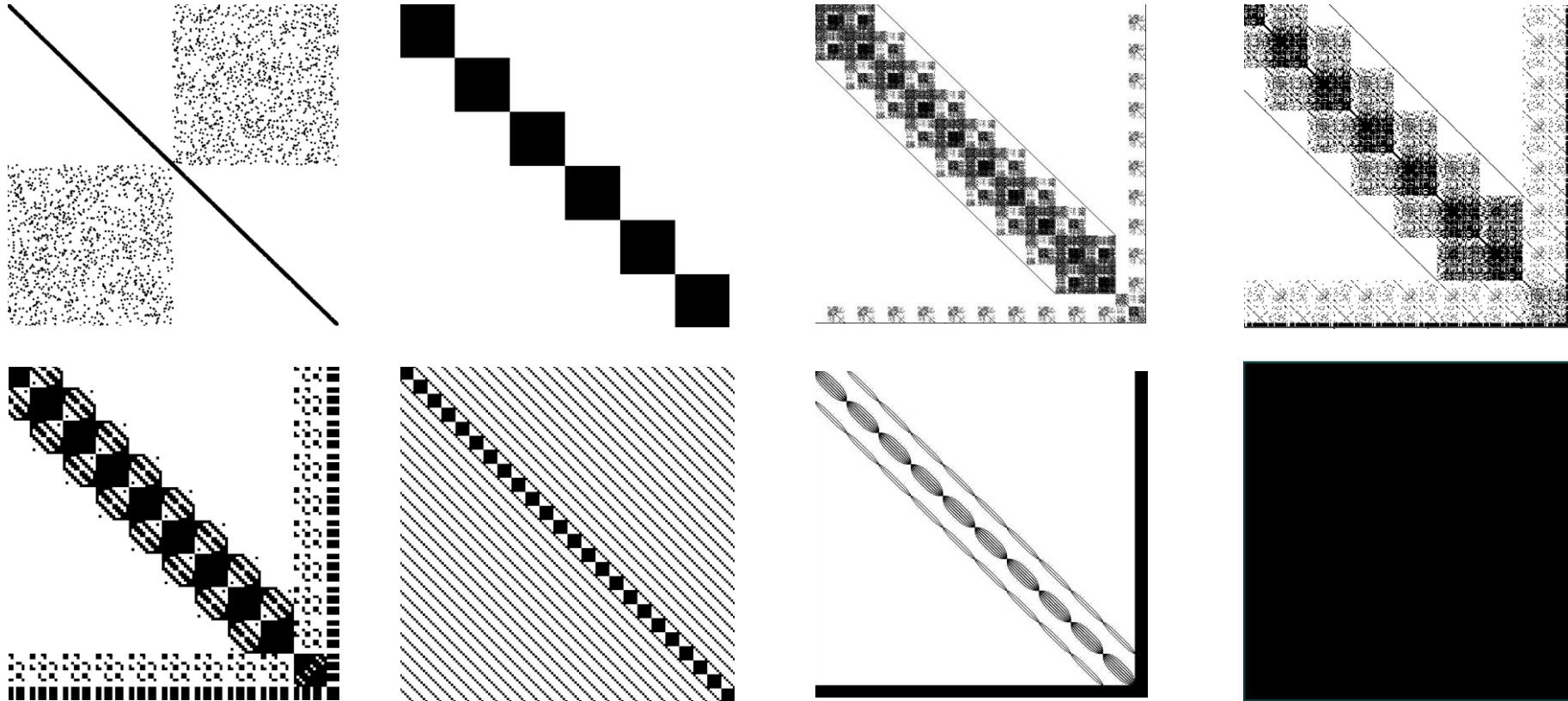
Real-World Application

- Modeling global temperature patterns using data from over 13,000 weather stations.
- Daily high/low temps for all of 2022 → nearly 5 million data records.
- Inference over ~400,000 random effects using INLA's efficient framework.

Structured Matrices in Bayesian Computation

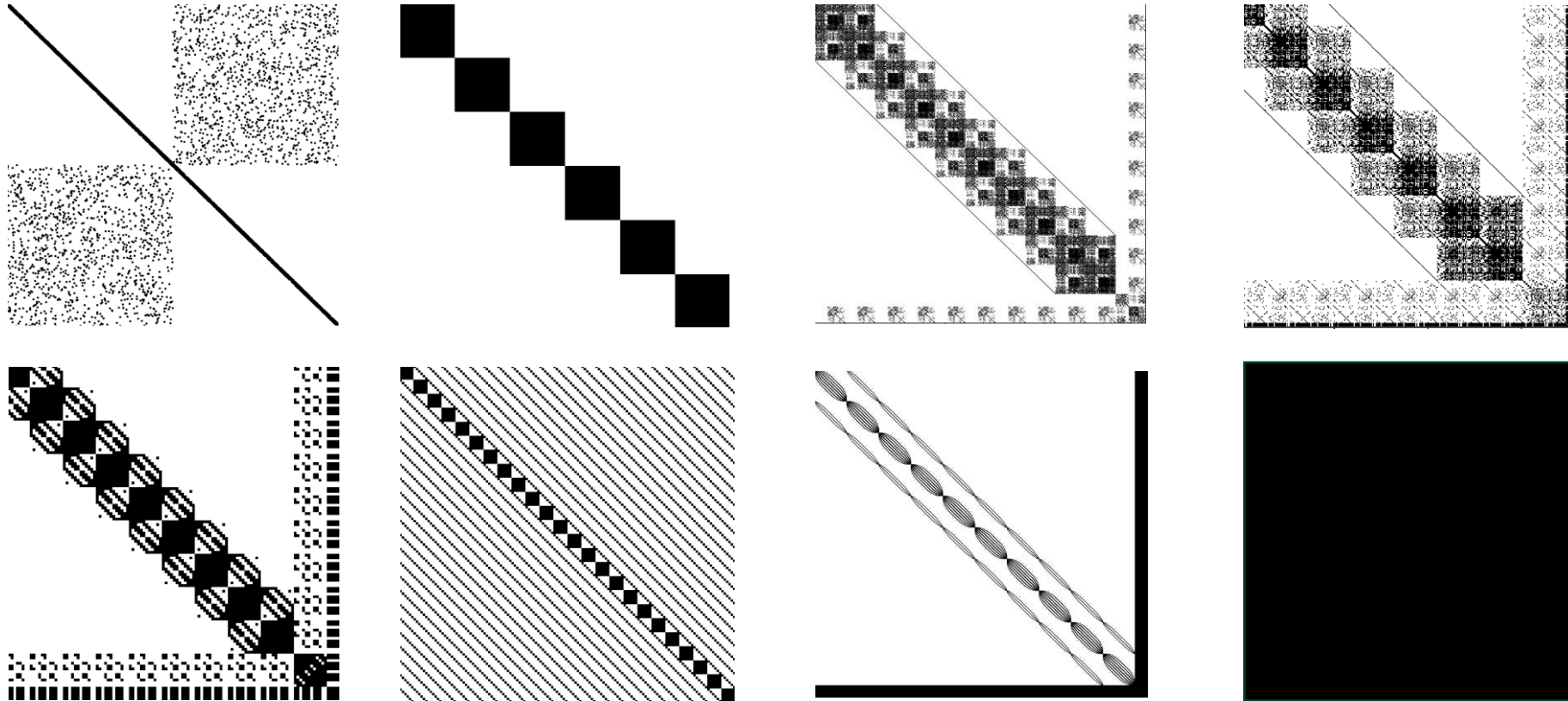


Structured Matrices in Bayesian Computation



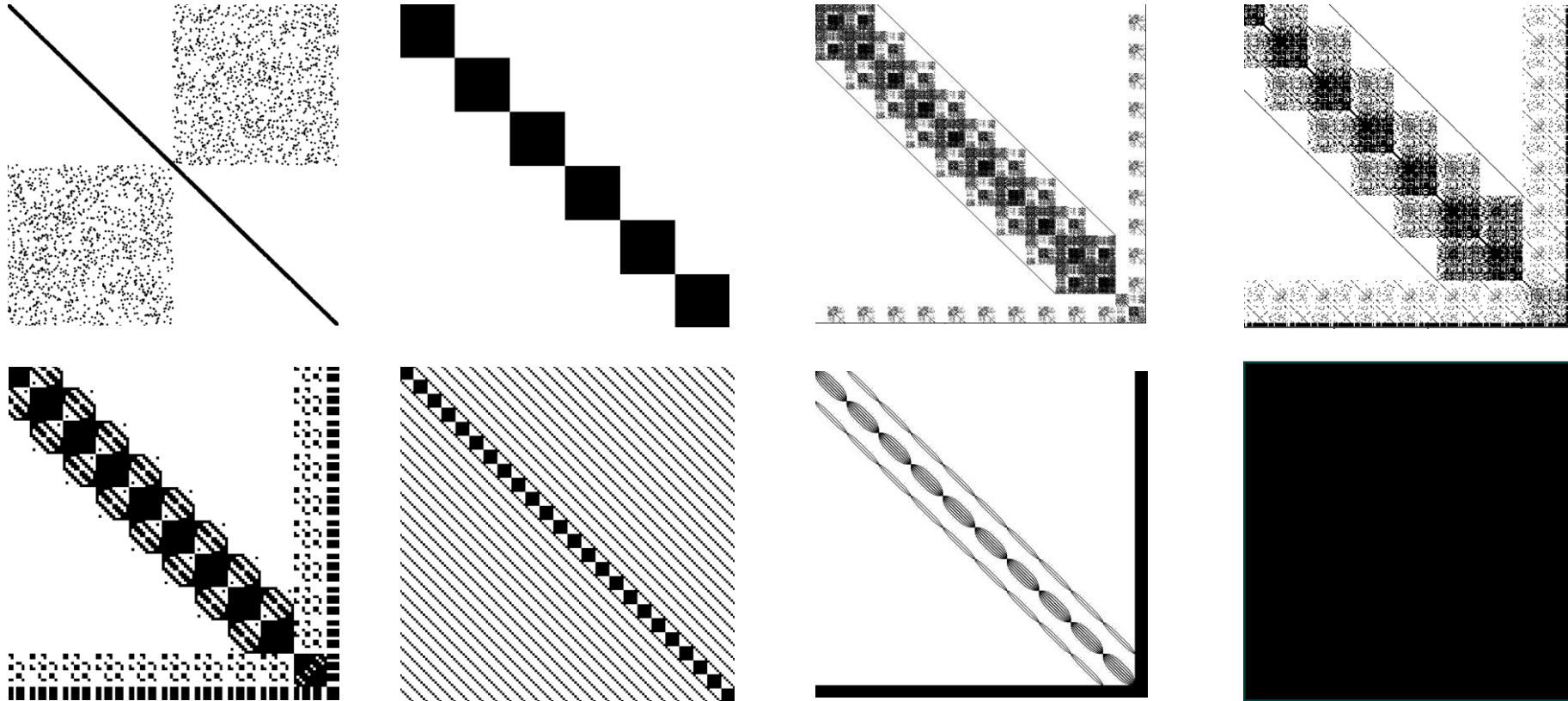
- Real-world applications (e.g., INLA) yield structured precision matrices like these.

Structured Matrices in Bayesian Computation



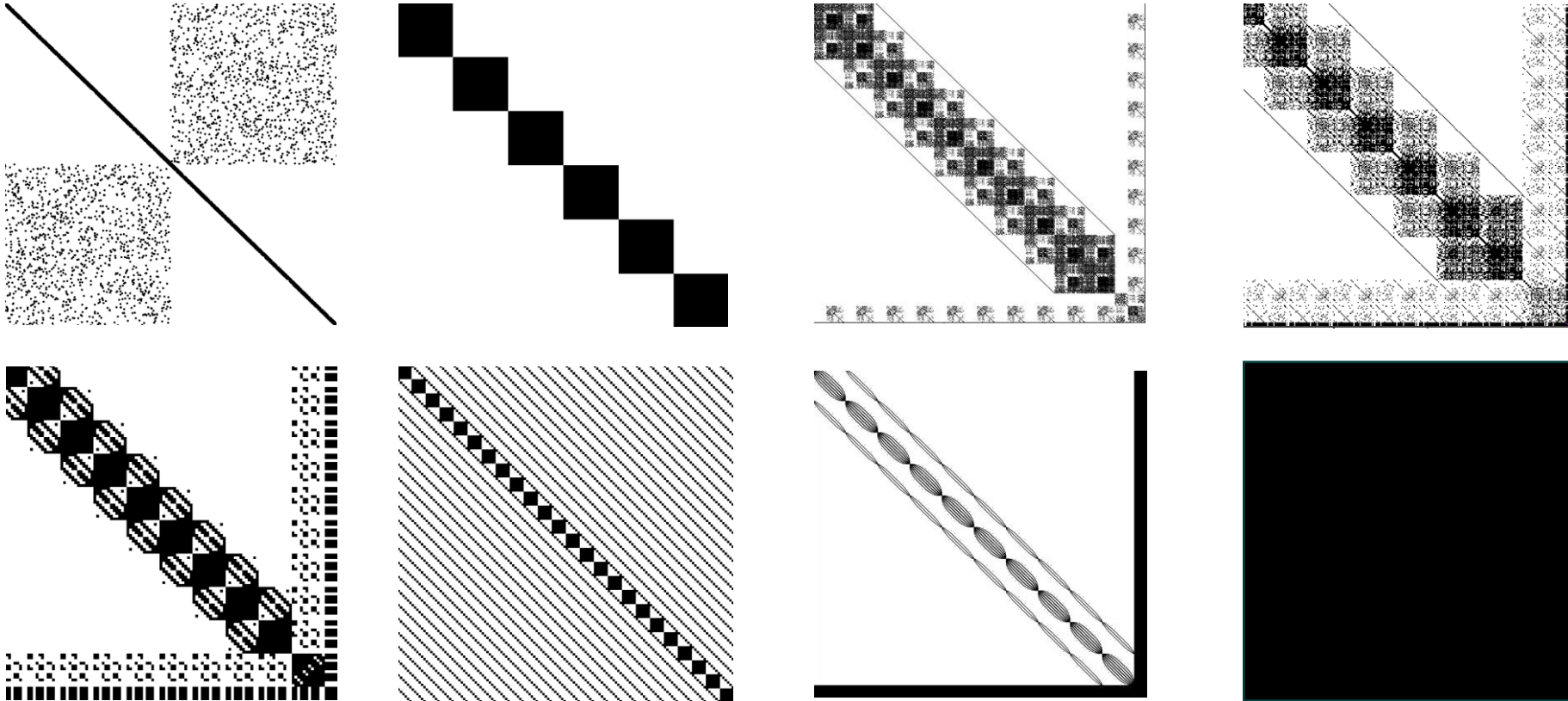
- Real-world applications (e.g., INLA) yield structured precision matrices like these.
- Patterns include: block diagonal, arrowhead, banded, and clustered dense tiles.

Structured Matrices in Bayesian Computation



- Many are not well suited for traditional sparse or dense solvers.

Structured Matrices in Bayesian Computation



- Many are not well suited for traditional sparse or dense solvers.
- Heterogeneous structure (all from the same INLA method) → poor performance with one-size-fits-all solvers.



The examples clearly demonstrate the need for a new scalable, high-performance framework that,





The examples clearly demonstrate the need for a new scalable, high-performance framework that, while not fully general, is designed to efficiently handle both sparse and dense computations across diverse matrix structures.





sTiles



sTiles



- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.

sTiles



- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.
- Designed for matrices that are not purely sparse, sTiles adapts to dense regions while efficiently handling moderate sparsity.



sTiles

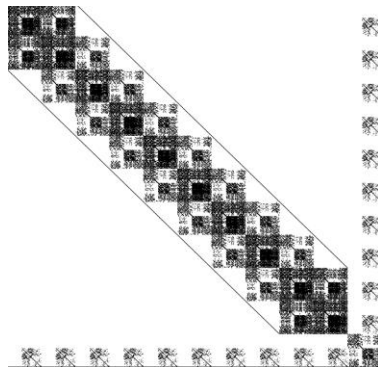


- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.
 - Designed for matrices that are not purely sparse, sTiles adapts to dense regions while efficiently handling moderate sparsity.
 - It converts a sparse matrix into a dense-structured tile matrix, facilitating the application of dense subroutines for matrix operations.
- 
- 
-

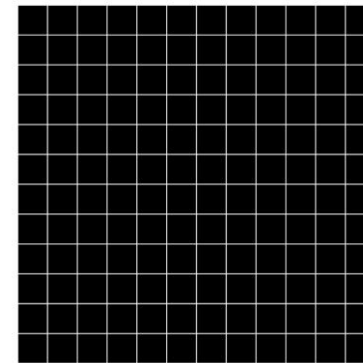
sTiles



- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.
- Designed for matrices that are not purely sparse, sTiles adapts to dense regions while efficiently handling moderate sparsity.
- It converts a sparse matrix into a dense-structured tile matrix, facilitating the application of dense subroutines for matrix operations.



sparse

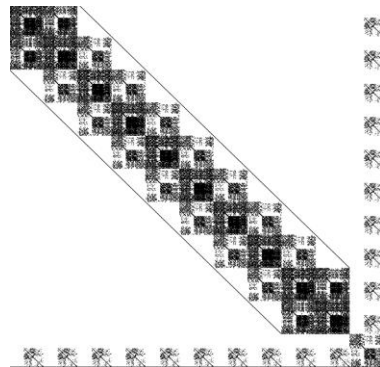


dense

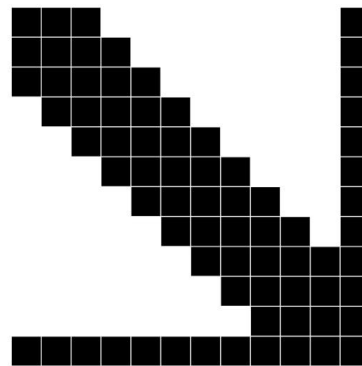
sTiles



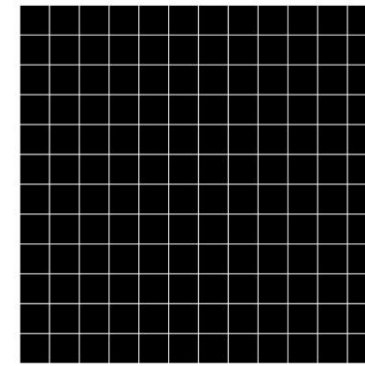
- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.
- Designed for matrices that are not purely sparse, sTiles adapts to dense regions while efficiently handling moderate sparsity.
- It converts a sparse matrix into a dense-structured tile matrix, facilitating the application of dense subroutines for matrix operations.



sparse



sTiles



dense



Dense

Sparse

Stores all values

Storage

Stores only non-zero values





Dense

Stores all values

sTiles



Storage

Stores only non-zero tiles

Sparse

Stores only non-zero values



Dense

Stores all values

Faster for small, dense data

sTiles



Storage

Stores only non-zero tiles

Speed

Sparse

Stores only non-zero values

Faster for large, mostly-zero data



Dense

Stores all values

Faster for small, dense data

sTiles



Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Sparse

Stores only non-zero values

Faster for large, mostly-zero data





Dense

sTiles



Sparse

Stores all values

Faster for small, dense data

High

Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Stores only non-zero values

Faster for large, mostly-zero data

Very low



Dense

sTiles



Sparse

Stores all values

Faster for small, dense data

High

Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Balanced

Stores only non-zero values

Faster for large, mostly-zero data

Very low



Dense

sTiles



Sparse

Stores all values

Faster for small, dense data

High

Regular

Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Balanced

Access Pattern

Stores only non-zero values

Faster for large, mostly-zero data

Very low

Irregular



Dense

Stores all values

Faster for small, dense data

High

Regular

sTiles



Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Balanced

Access Pattern

Mostly regular within tiles

Sparse

Stores only non-zero values

Faster for large, mostly-zero data

Very low

Irregular



Dense

sTiles



Sparse

Stores all values

Storage

Stores only non-zero tiles

Stores only non-zero values

Faster for small, dense data

Speed

Fast for structured sparsity

Faster for large, mostly-zero data

High

Memory Use

Balanced

Very low

Regular

Access Pattern

Mostly regular within tiles

Irregular

Easy

Parallelism

Difficult



Dense

sTiles



Sparse

Stores all values

Faster for small, dense data

High

Regular

Easy

Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Balanced

Access Pattern

Mostly regular within tiles

Parallelism

Efficient

Stores only non-zero values

Faster for large, mostly-zero data

Very low

Irregular

Difficult



Dense

sTiles



Sparse

Stores all values

Faster for small, dense data

High

Regular

Easy

Very efficient

Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Balanced

Access Pattern

Mostly regular within tiles

Parallelism

Efficient

GPU-Friendly

Stores only non-zero values

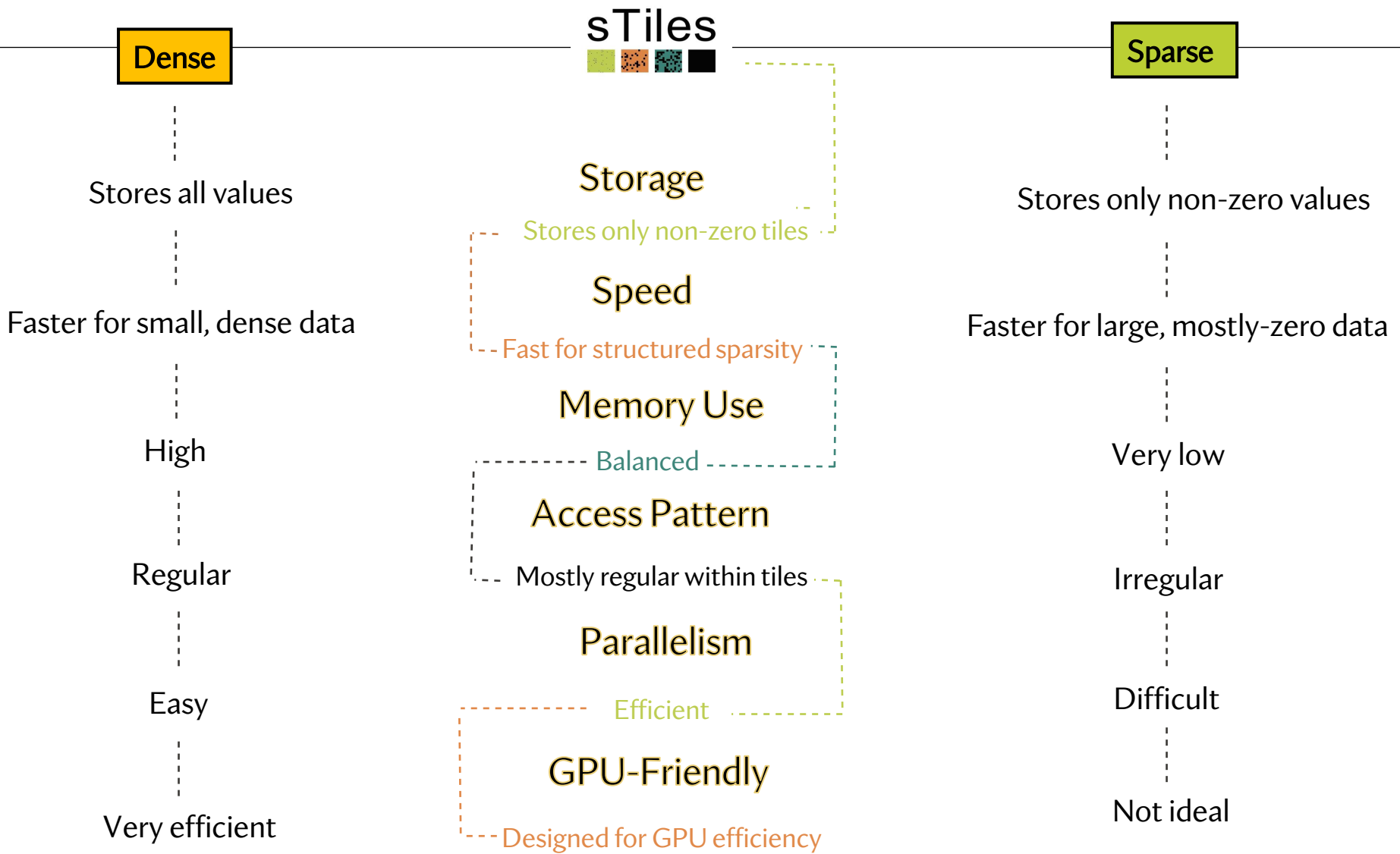
Faster for large, mostly-zero data

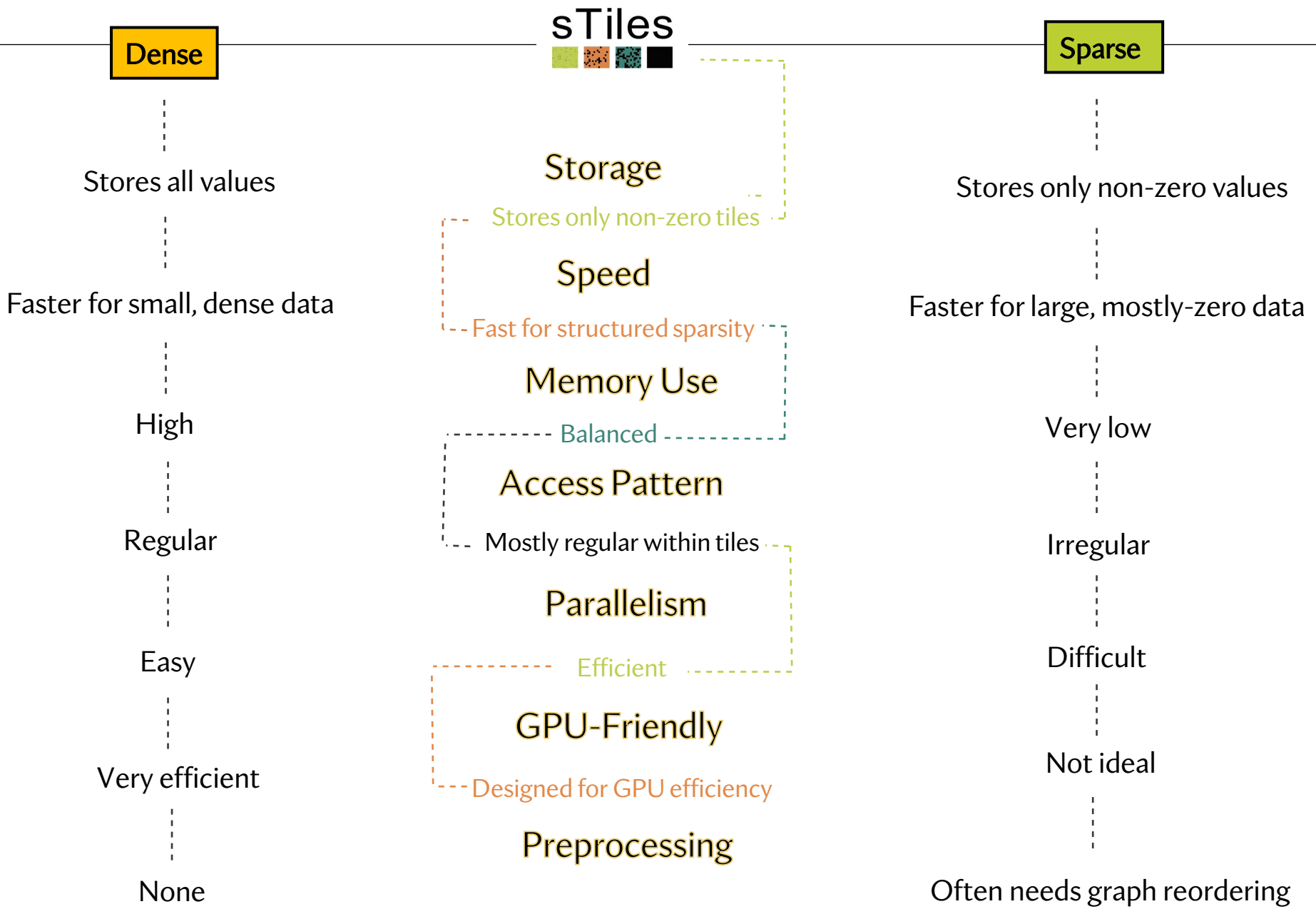
Very low

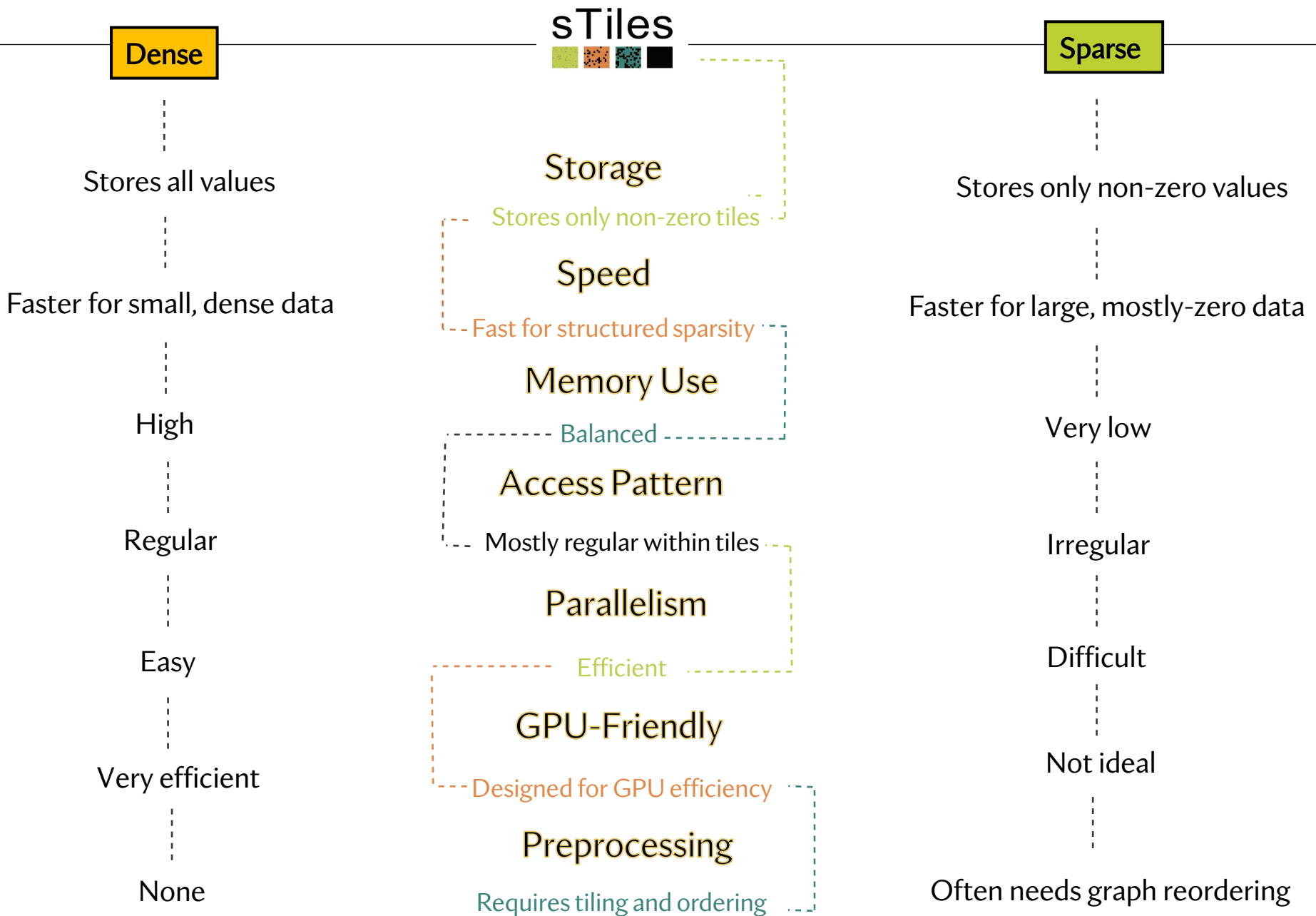
Irregular

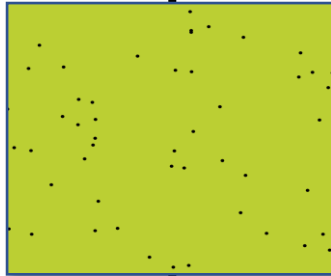
Difficult

Not ideal









I. Arrowhead Sparse Matrix Factorization

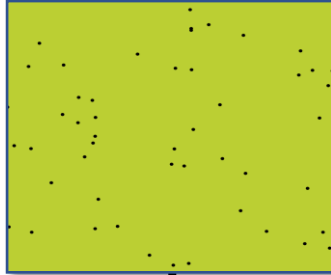
- Permutation Techniques
- Compressed Tile Storage Format (CTSF)
- Sparse Cholesky Algorithm using Static Scheduling

II. Parallel Optimization Techniques and GPU Acceleration

III. Performance Comparison and Experimental Results

IV. Conclusion and Outlook





I. Arrowhead Sparse Matrix Factorization

- Permutation Techniques
 - Compressed Tile Storage Format (CTSF)
 - Sparse Cholesky Algorithm using Static Scheduling

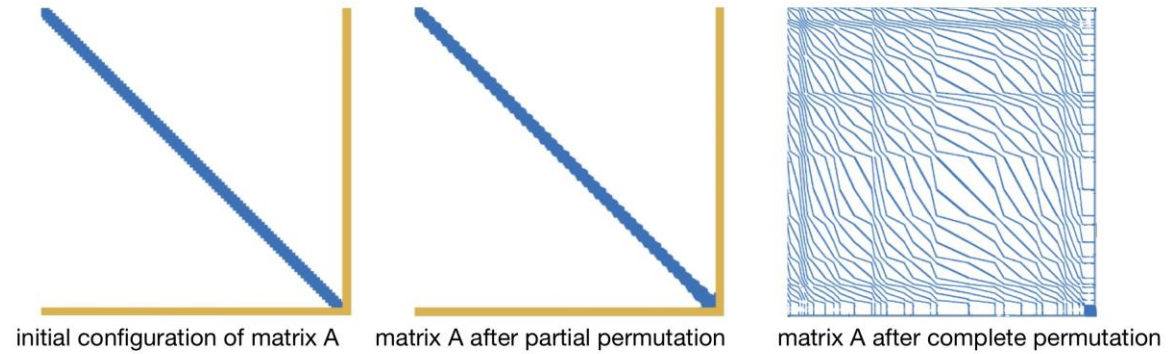
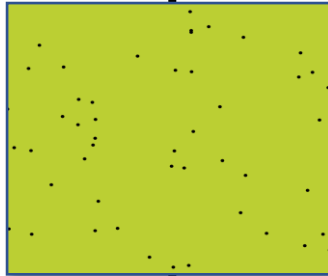
II. Parallel Optimization Techniques and GPU Acceleration

III. Performance Comparison and Experimental Results

IV. Conclusion and Outlook



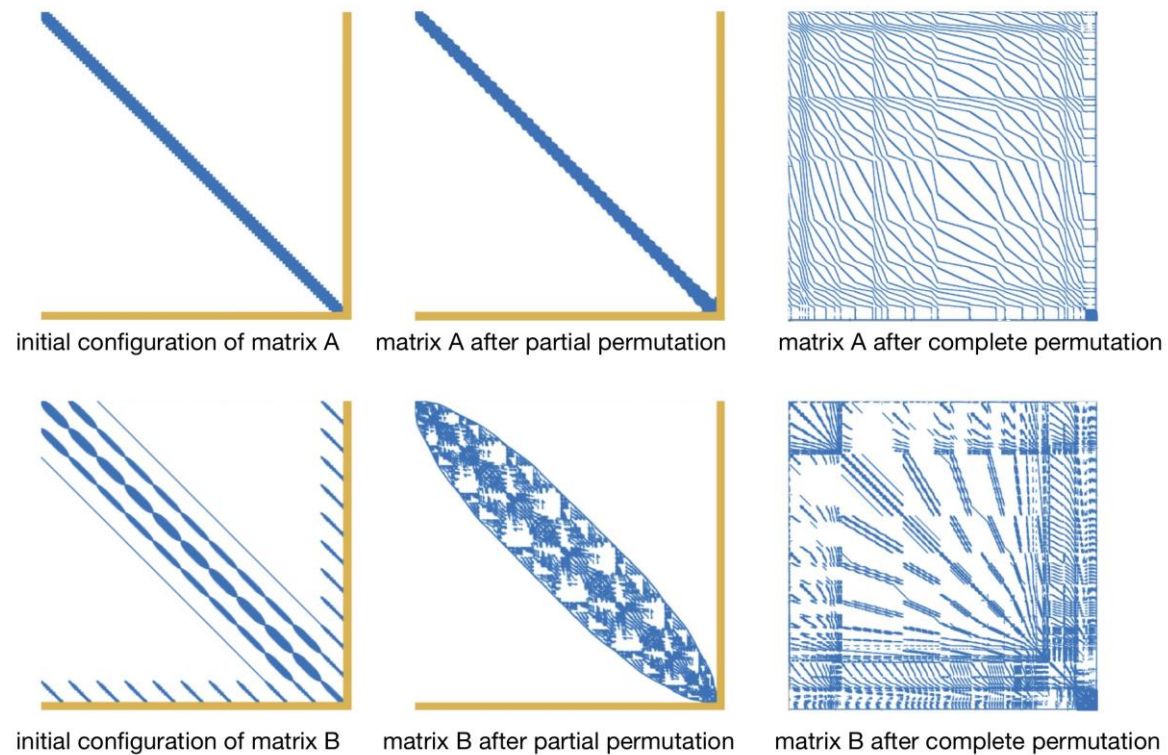
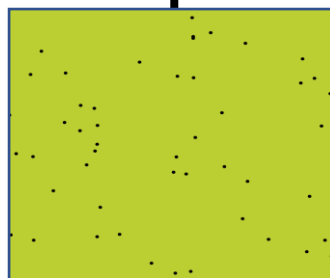
Reverse Cuthill-McKee



- Reduces matrix bandwidth by reordering nodes to cluster nonzeros closer to the diagonal



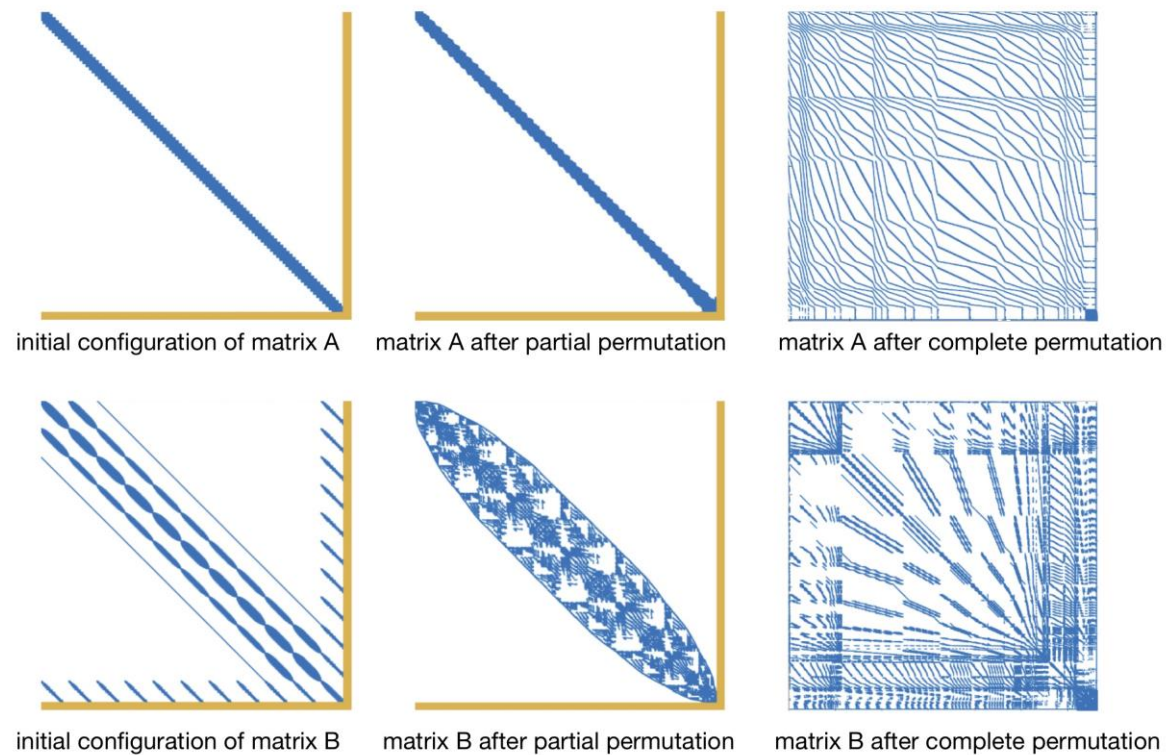
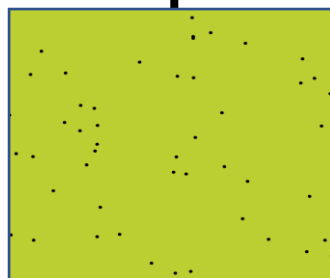
Reverse Cuthill-McKee



- Reduces matrix bandwidth by reordering nodes to cluster nonzeros closer to the diagonal



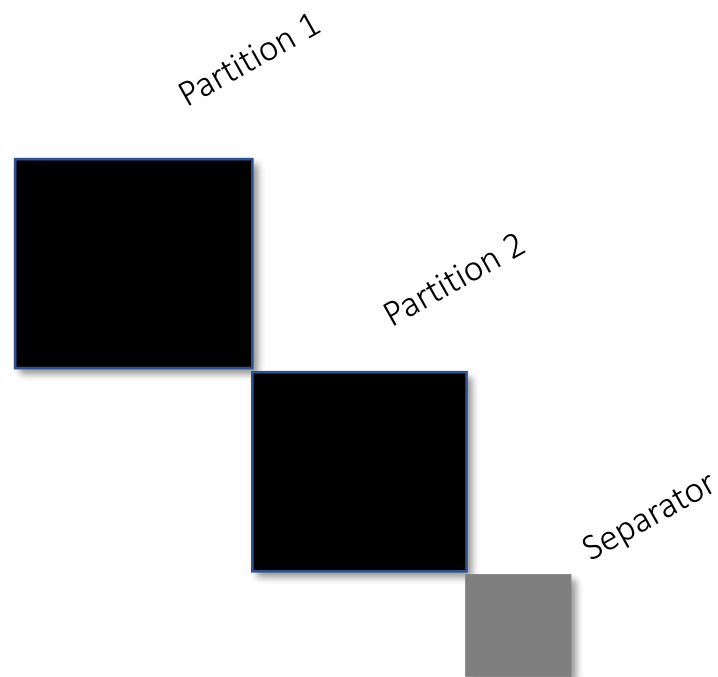
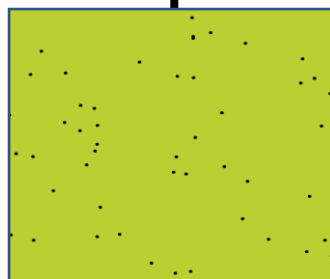
Reverse Cuthill-McKee



- In Matrix B: partial RCM → 32.7% decrease in fill-in vs original



Nested Dissection

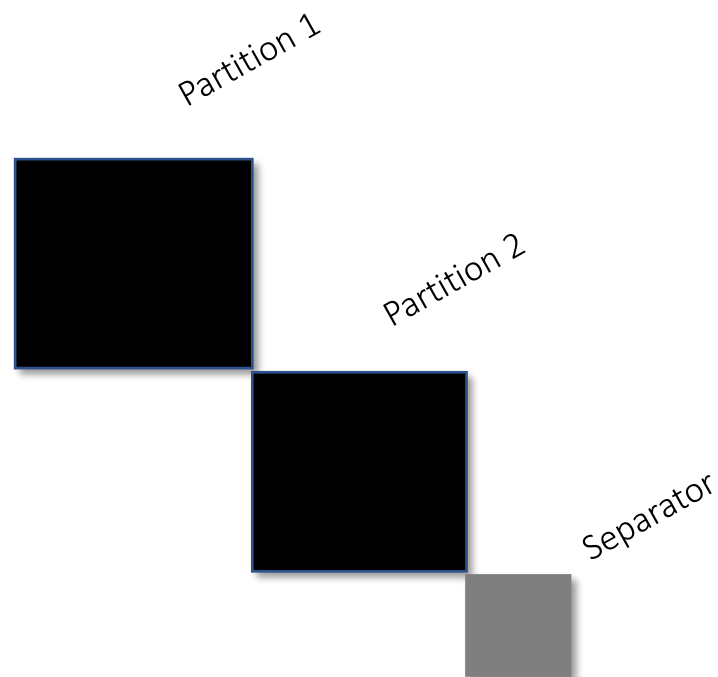
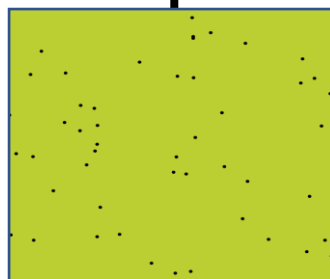


Level 1 – Nested Dissection

- Recursively partitions matrix graph using vertex separators



Nested Dissection

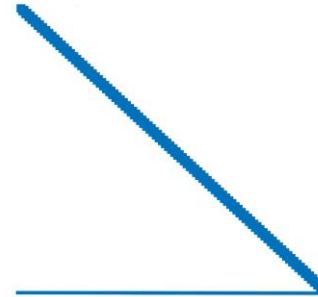
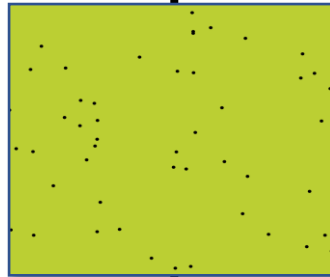


Level 1 – Nested Dissection

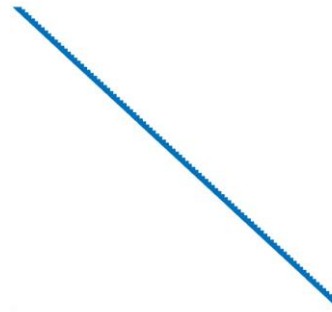
- Enables parallelism by processing independent blocks



Nested Dissection



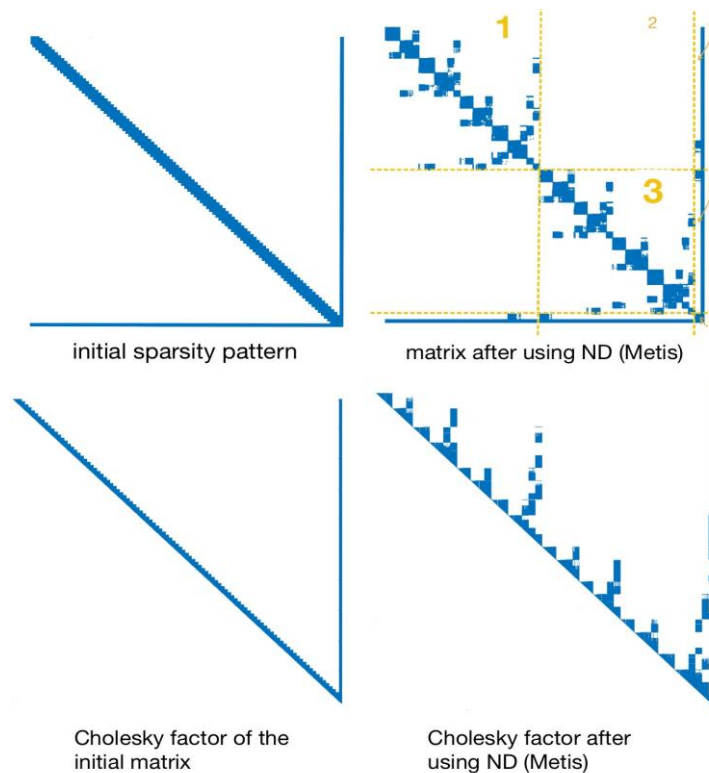
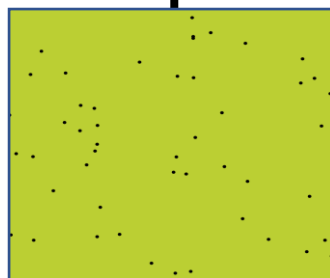
initial sparsity pattern



Cholesky factor of the
initial matrix



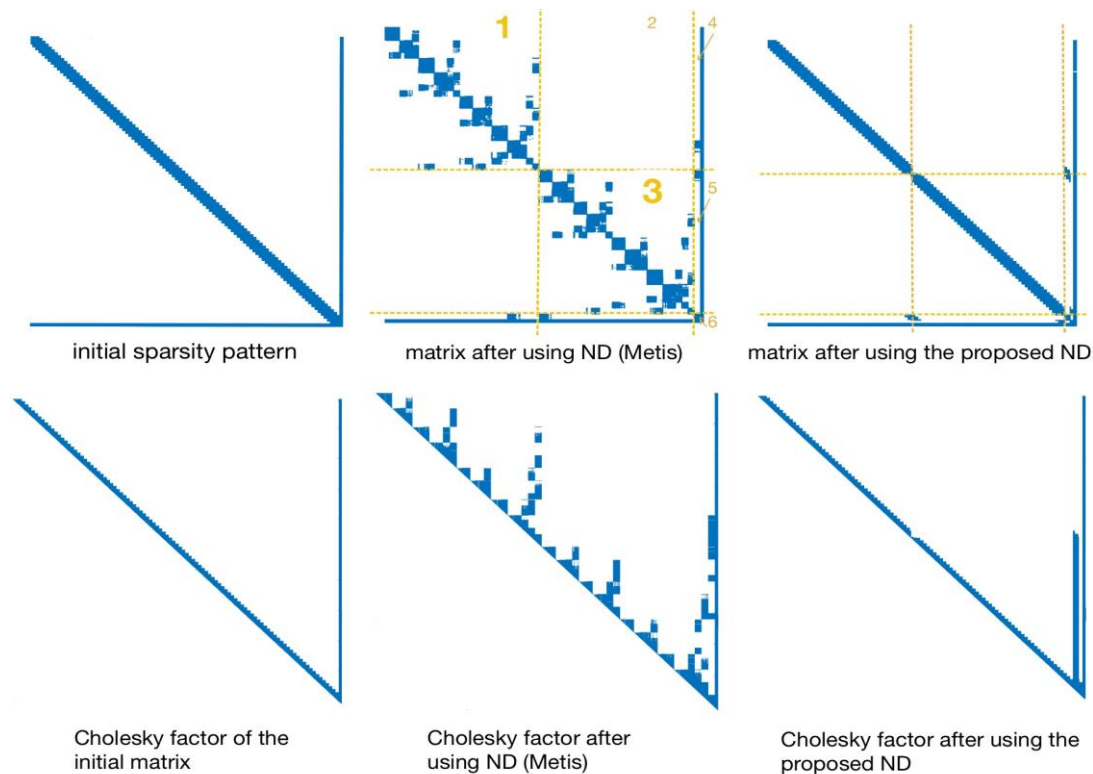
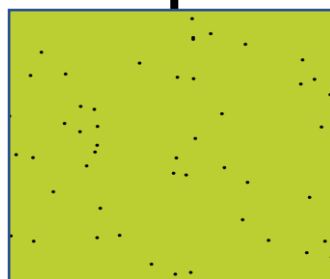
Nested Dissection



- Standard ND (e.g., METIS) often destroys arrowhead structure

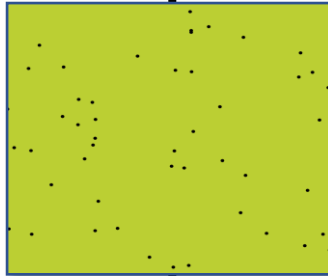


Nested Dissection



- We propose an adaptable ND that improves parallelism





I. Arrowhead Sparse Matrix Factorization

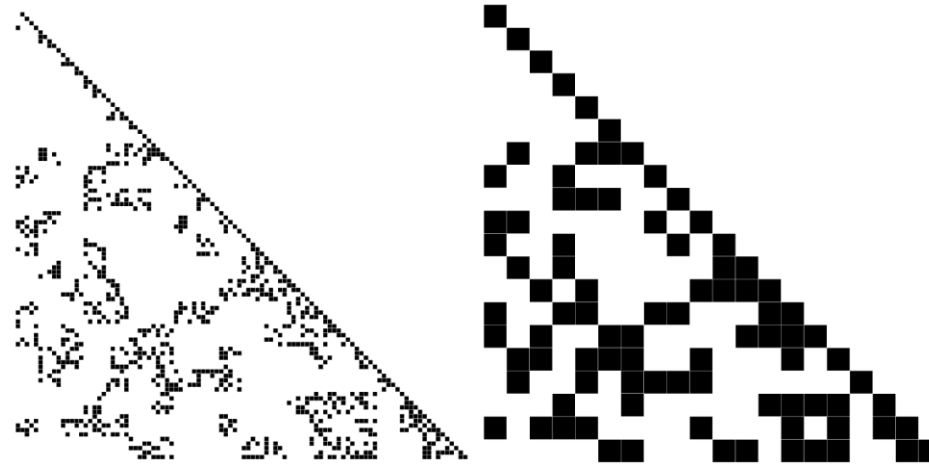
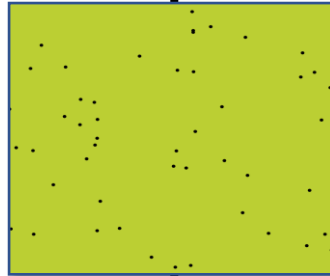
- Permutation Techniques
- Compressed Tile Storage Format (CTSF)
- Sparse Cholesky Algorithm using Static Scheduling

II. Parallel Optimization Techniques and GPU Acceleration

III. Performance Comparison and Experimental Results

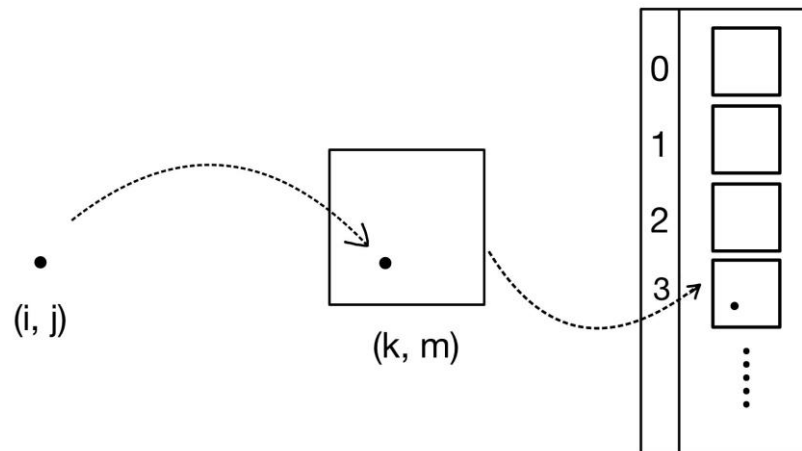
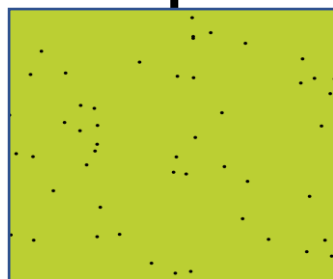
IV. Conclusion and Outlook





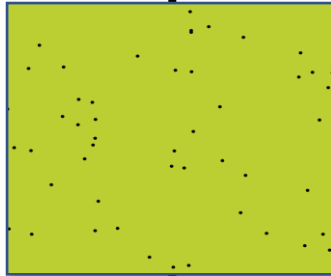
- Sparse Matrix Conversion: converts a sparse matrix into a dense-structured tile matrix, facilitating the application of dense subroutines for matrix operations.





- Element-wise mapping translates a sparse matrix into a dense tile structure by assigning nonzeros to tile coordinates, which are stored in contiguous memory blocks to enable cache-friendly access and fine-grained parallelism.





I. Arrowhead Sparse Matrix Factorization

- Permutation Techniques
- Compressed Tile Storage Format (CTSF)
- Sparse Cholesky Algorithm using Static Scheduling

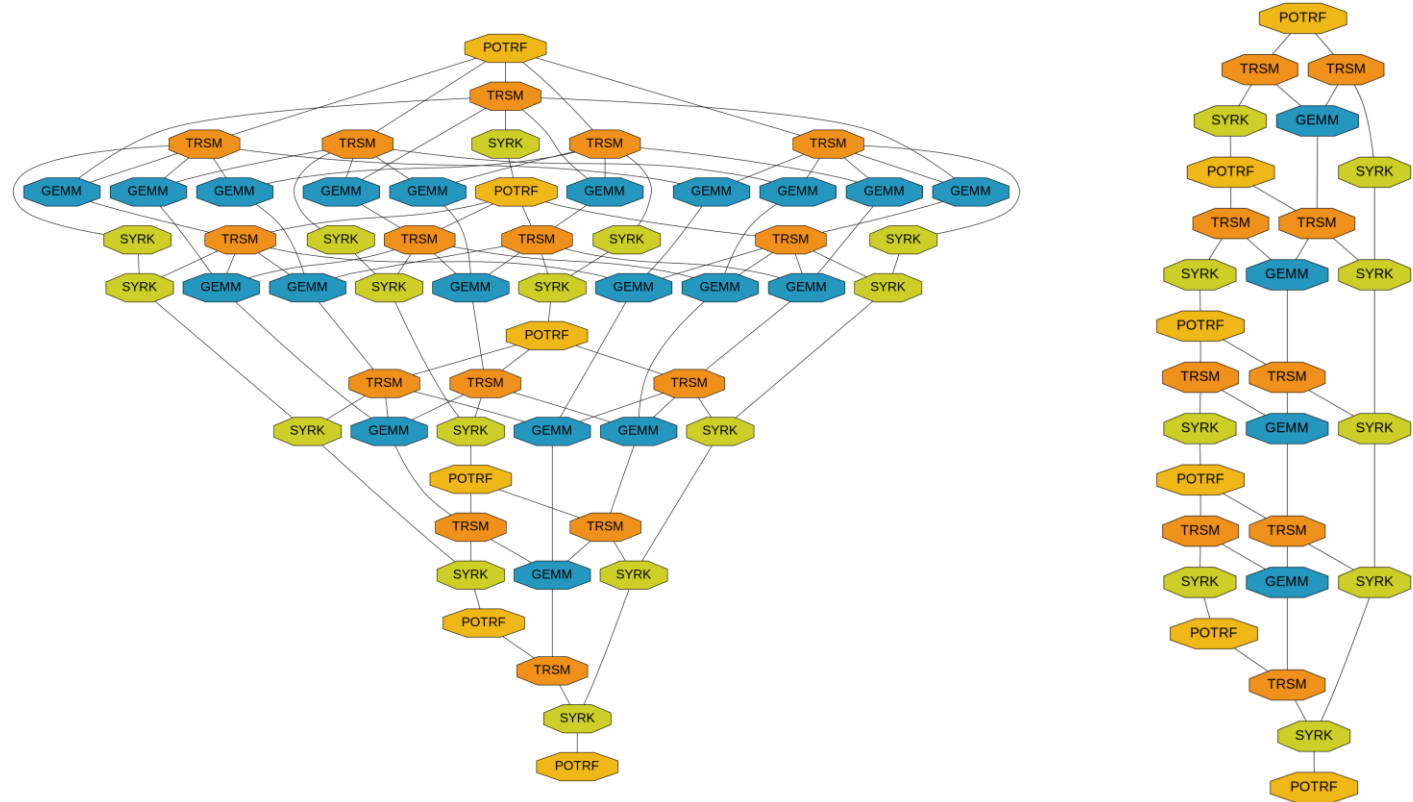
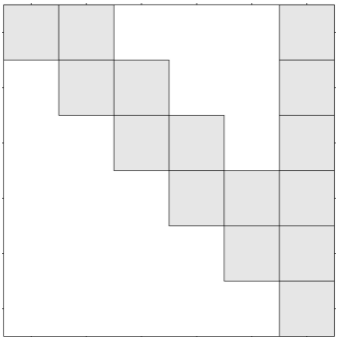
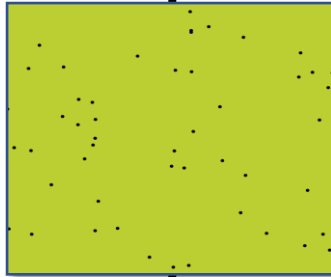
II. Parallel Optimization Techniques and GPU Acceleration

III. Performance Comparison and Experimental Results

IV. Conclusion and Outlook

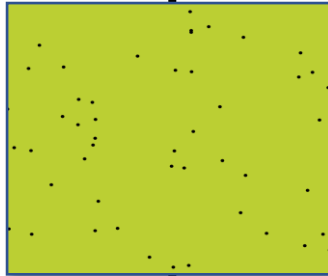


Directed Acyclic Graph



- The factorization process is represented as a Directed Acyclic Graph (DAG), where each node is a computational task and edges represent data dependencies between tiles.

Asynchronous Task-based Sparse Cholesky Factorization

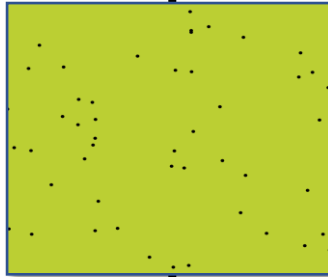


- Tasks are pre-assigned to threads using a Task Assignment Table (TAT), avoiding runtime scheduling overhead.

```
for each thread ID do
  for each task i assigned to thread ID do
    m = TAT[i, 0]
    k = TAT[i, 1]
    n = TAT[i, 2]
    task_type = TAT[i, 3]
    if task_type = 1 then
       $A_{kk} \leftarrow \text{POTRF}(A_{kk})$ 
      Set core_progress[k,k] = 1
    else if task_type = 2 then
      while core_progress[n,k]  $\neq$  1 do
        /* Wait */
      end while
       $A_{kk} \leftarrow \text{SYRK}(A_{nk}^T, A_{kk})$ 
    else if task_type = 3 then
      while core_progress[k,k]  $\neq$  1 do
        /* Wait */
      end while
       $A_{km} \leftarrow \text{TRSM}(A_{kk}, A_{km})$ 
      Set core_progress[m,k] = 1
    else if task_type = 4 then
      while core_progress[k,n]  $\neq$  1 do
        /* Wait */
      end while
      while core_progress[m,n]  $\neq$  1 do
        /* Wait */
      end while
       $A_{km} \leftarrow \text{GEMM}(A_{nk}^T, A_{mn}, A_{km})$ 
    end if
  end for
end for
```



Asynchronous Task-based Sparse Cholesky Factorization

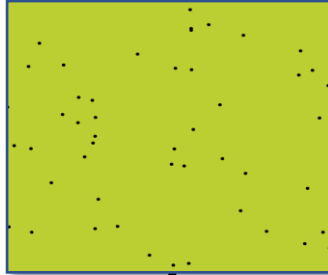


- Tasks are pre-assigned to threads using a Task Assignment Table (TAT), avoiding runtime scheduling overhead.
- A global progress table tracks task completion and enforces correct dependency order.

```
for each thread ID do
  for each task i assigned to thread ID do
    m = TAT[i, 0]
    k = TAT[i, 1]
    n = TAT[i, 2]
    task_type = TAT[i, 3]
    if task_type = 1 then
       $A_{kk} \leftarrow \text{POTRF}(A_{kk})$ 
      Set core_progress[k,k] = 1
    else if task_type = 2 then
      while core_progress[n,k]  $\neq$  1 do
        /* Wait */
      end while
       $A_{kk} \leftarrow \text{SYRK}(A_{nk}^T, A_{kk})$ 
    else if task_type = 3 then
      while core_progress[k,k]  $\neq$  1 do
        /* Wait */
      end while
       $A_{km} \leftarrow \text{TRSM}(A_{kk}, A_{km})$ 
      Set core_progress[m,k] = 1
    else if task_type = 4 then
      while core_progress[k,n]  $\neq$  1 do
        /* Wait */
      end while
      while core_progress[m,n]  $\neq$  1 do
        /* Wait */
      end while
       $A_{km} \leftarrow \text{GEMM}(A_{nk}^T, A_{mn}, A_{km})$ 
    end if
  end for
end for
```



Asynchronous Task-based Sparse Cholesky Factorization

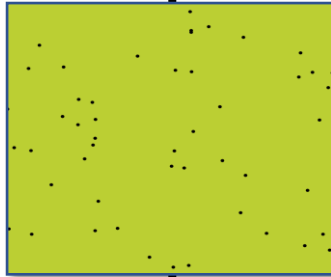


- Tasks are pre-assigned to threads using a Task Assignment Table (TAT), avoiding runtime scheduling overhead.
- A global progress table tracks task completion and enforces correct dependency order.
- Left-looking Cholesky variant allows accumulation via GEMMs, exposing more parallelism in sparse regions.

```
for each thread ID do
  for each task i assigned to thread ID do
    m = TAT[i, 0]
    k = TAT[i, 1]
    n = TAT[i, 2]
    task_type = TAT[i, 3]
    if task_type = 1 then
       $A_{kk} \leftarrow \text{POTRF}(A_{kk})$ 
      Set core_progress[k,k] = 1
    else if task_type = 2 then
      while core_progress[n,k]  $\neq$  1 do
        /* Wait */
      end while
       $A_{kk} \leftarrow \text{SYRK}(A_{nk}^T, A_{kk})$ 
    else if task_type = 3 then
      while core_progress[k,k]  $\neq$  1 do
        /* Wait */
      end while
       $A_{km} \leftarrow \text{TRSM}(A_{kk}, A_{km})$ 
      Set core_progress[m,k] = 1
    else if task_type = 4 then
      while core_progress[k,n]  $\neq$  1 do
        /* Wait */
      end while
      while core_progress[m,n]  $\neq$  1 do
        /* Wait */
      end while
       $A_{km} \leftarrow \text{GEMM}(A_{nk}^T, A_{mn}, A_{km})$ 
    end if
  end for
end for
```



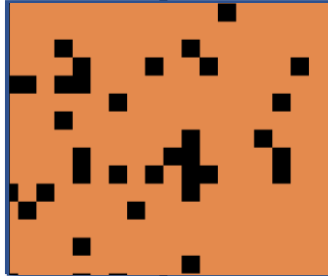
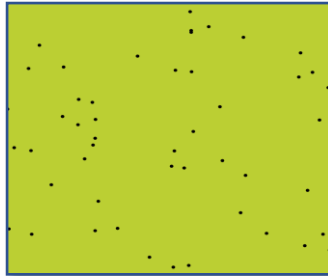
Asynchronous Task-based Sparse Cholesky Factorization



- Tasks are pre-assigned to threads using a Task Assignment Table (TAT), avoiding runtime scheduling overhead.
- A global progress table tracks task completion and enforces correct dependency order.
- Left-looking Cholesky variant allows accumulation via GEMMs, exposing more parallelism in sparse regions.
- Static pipeline scheduling enables asynchronous task progression and better cache locality.

```
for each thread ID do
  for each task i assigned to thread ID do
    m = TAT[i, 0]
    k = TAT[i, 1]
    n = TAT[i, 2]
    task_type = TAT[i, 3]
    if task_type = 1 then
       $A_{kk} \leftarrow \text{POTRF}(A_{kk})$ 
      Set core_progress[k,k] = 1
    else if task_type = 2 then
      while core_progress[n,k]  $\neq$  1 do
        /* Wait */
      end while
       $A_{kk} \leftarrow \text{SYRK}(A_{nk}^T, A_{kk})$ 
    else if task_type = 3 then
      while core_progress[k,k]  $\neq$  1 do
        /* Wait */
      end while
       $A_{km} \leftarrow \text{TRSM}(A_{kk}, A_{km})$ 
      Set core_progress[m,k] = 1
    else if task_type = 4 then
      while core_progress[k,n]  $\neq$  1 do
        /* Wait */
      end while
      while core_progress[m,n]  $\neq$  1 do
        /* Wait */
      end while
       $A_{km} \leftarrow \text{GEMM}(A_{nk}^T, A_{mn}, A_{km})$ 
    end if
  end for
end for
```





I. Arrowhead Sparse Matrix Factorization

II. Parallel Optimization Techniques and GPU Acceleration

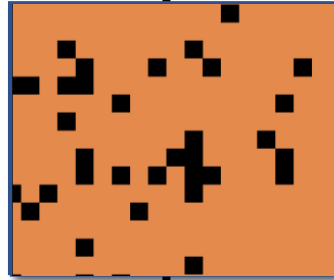
- Tree Reduction
- GPU Implementation

III. Performance Comparison and Experimental Results

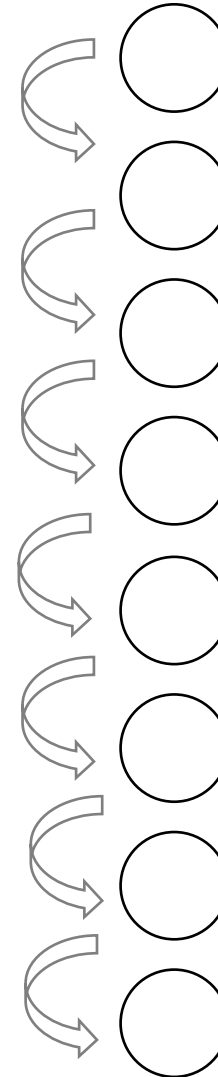
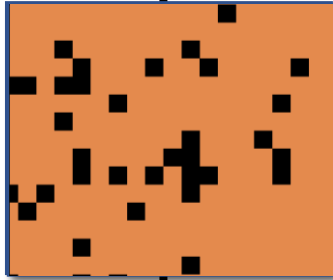
IV. Conclusion and Outlook



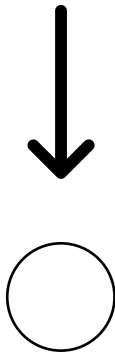
- Weaken the synchronization points due to Read-After-Write data hazards using a tree reduction for the GEMM accumulation.



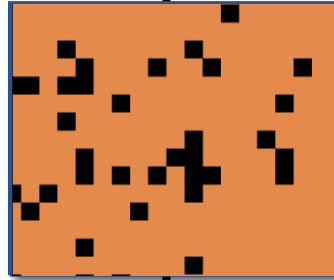
- Weaken the synchronization points due to Read-After-Write data hazards using a tree reduction for the GEMM accumulation.



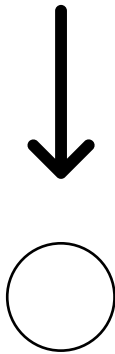
- Weaken the synchronization points due to Read-After-Write data hazards using a tree reduction for the GEMM accumulation.
- GEMM result on tile A is saved at



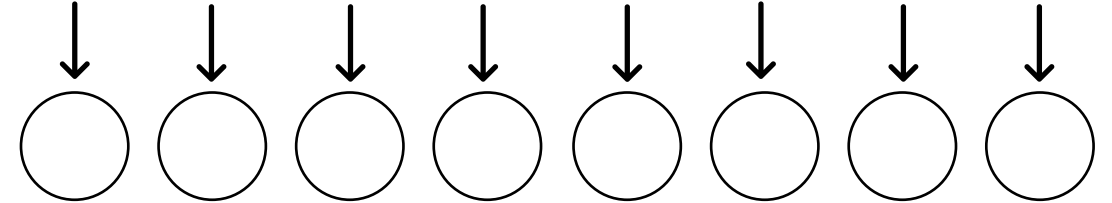
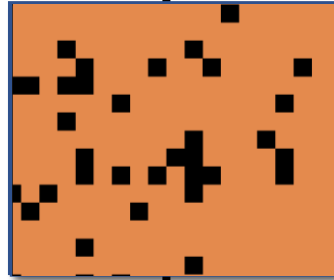
1/2



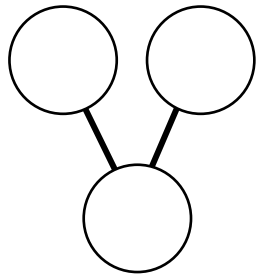
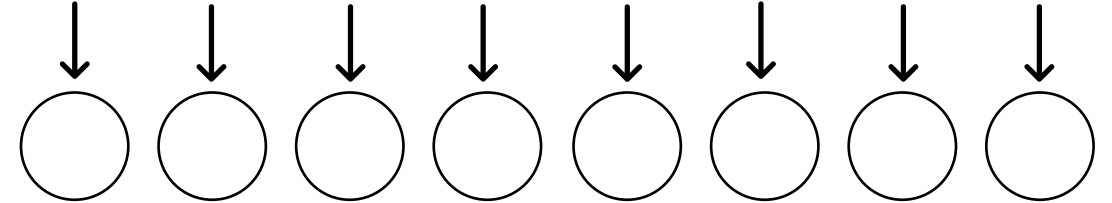
- Weaken the synchronization points due to Read-After-Write data hazards using a tree reduction for the GEMM accumulation.
- GEMM result on tile A is saved at



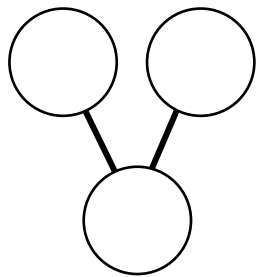
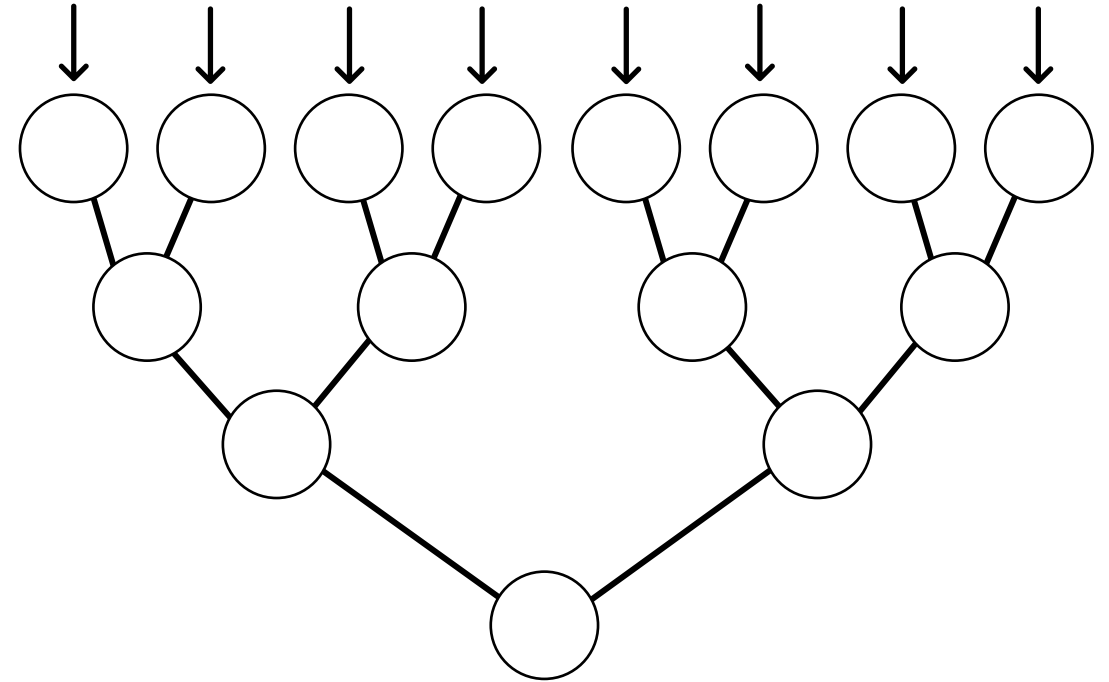
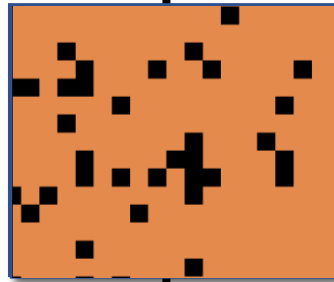
1/2



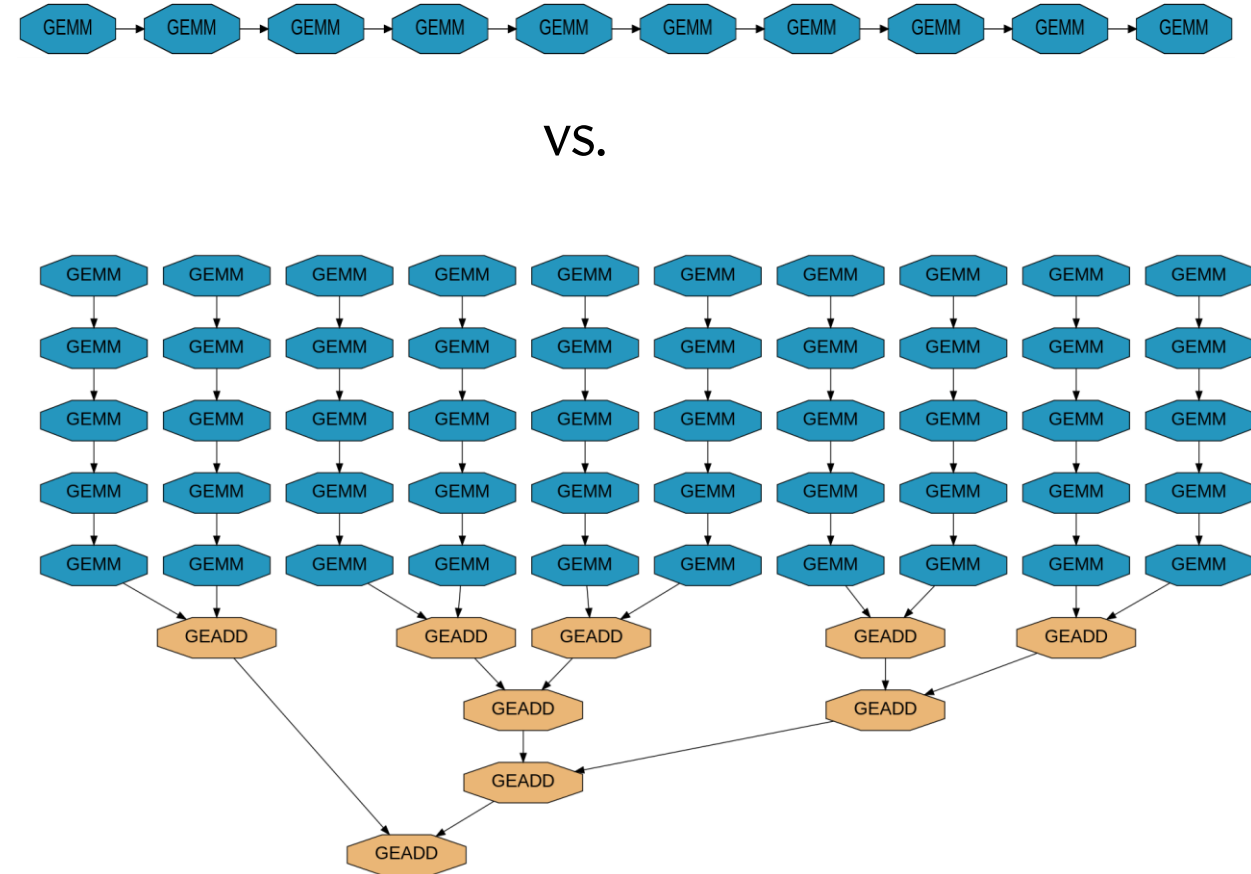
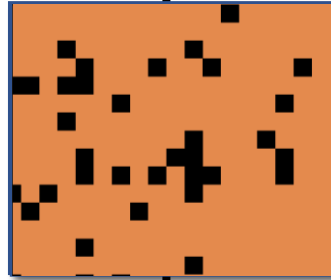
- Weaken the synchronization points due to Read-After-Write data hazards using a tree reduction for the GEMM accumulation.
- add the results of two GEMMs

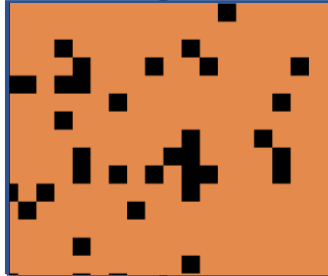
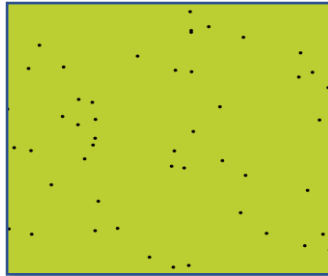


- Weaken the synchronization points due to Read-After-Write data hazards using a tree reduction for the GEMM accumulation.
- add the results of two GEMMs



- Weaken the synchronization points due to Read-After-Write data hazards using a tree reduction for the GEMM accumulation.





I. Arrowhead Sparse Matrix Factorization

II. Parallel Optimization Techniques and GPU Acceleration

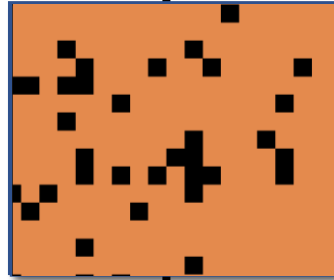
- Tree Reduction
- GPU Implementation

III. Performance Comparison and Experimental Results

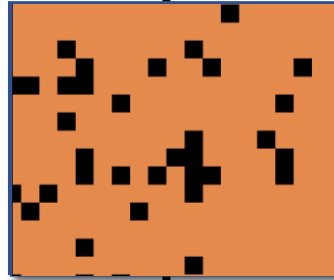
IV. Conclusion and Outlook



- Each thread is assigned a separate CUDA stream to enable concurrent execution of tasks on GPU.



- Each thread is assigned a separate CUDA stream to enable concurrent execution of tasks on GPU.
- Dense tile operations are offloaded using cuBLAS/cuSOLVER kernels (e.g., cublasDgemm, cusolverDnDpotrf).



- Each thread is assigned a separate CUDA stream to enable concurrent execution of tasks on GPU.
- Dense tile operations are offloaded using cuBLAS/cuSOLVER kernels (e.g., cublasDgemm, cusolverDnDpotrf).
- Tile size is increased on GPU (e.g., 600) to improve hardware occupancy and throughput.



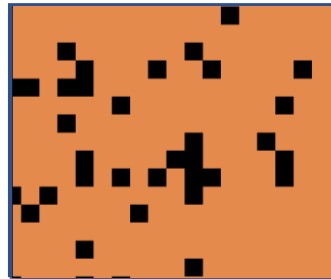
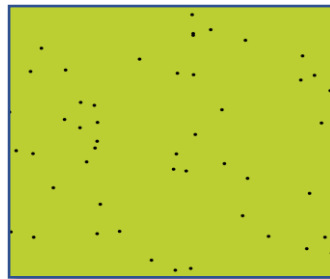
- Each thread is assigned a separate CUDA stream to enable concurrent execution of tasks on GPU.
- Dense tile operations are offloaded using cuBLAS/cuSOLVER kernels (e.g., cublasDgemm, cusolverDnDpotrf).
- Tile size is increased on GPU (e.g., 600) to improve hardware occupancy and throughput.

- Current version supports in-core GPU execution; out-of-core and multi-GPU extensions are part of future work.



GPU





I. Arrowhead Sparse Matrix Factorization

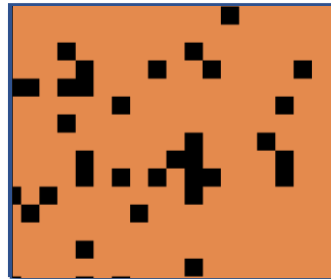
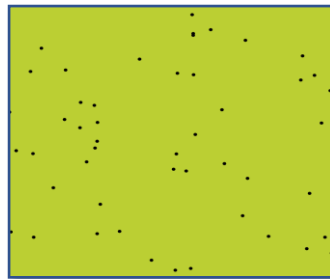
II. Parallel Optimization Techniques and GPU Acceleration

III. Performance Comparison and Experimental Results

- Cross-Architecture Performance Comparison
- Scalability Analysis
- Tree Reduction Performance
- GPU Acceleration Example

IV. Conclusion and Outlook





I. Arrowhead Sparse Matrix Factorization

II. Parallel Optimization Techniques and GPU Acceleration

III. Performance Comparison and Experimental Results

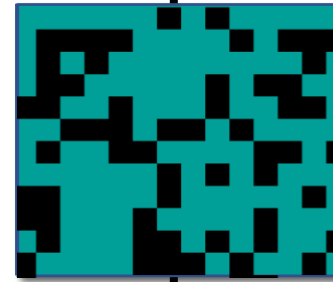
- Cross-Architecture Performance Comparison
- Scalability Analysis
- Tree Reduction Performance
- GPU Acceleration Example

IV. Conclusion and Outlook



Experimental Setup: benchmarks were run on two high-performance servers:

- **Server 1:** Dual-socket Intel Xeon Gold 6230R (Cascade Lake), 52 cores total, 71.5MB L3 cache.
- **Server 2:** Dual-socket AMD EPYC 7713 (Milan), 128 cores total, 512MB L3 cache.



Both systems used to evaluate scalability and performance across diverse architectures.

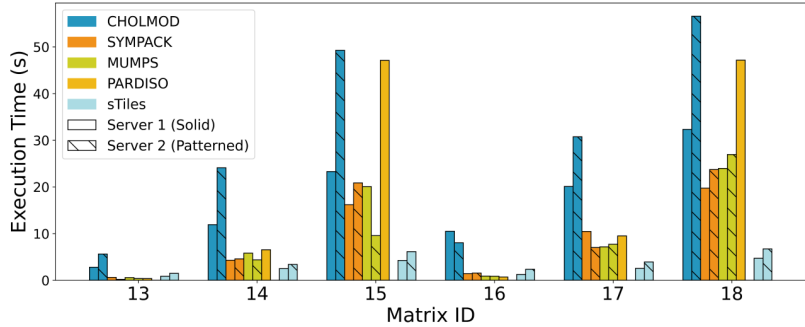
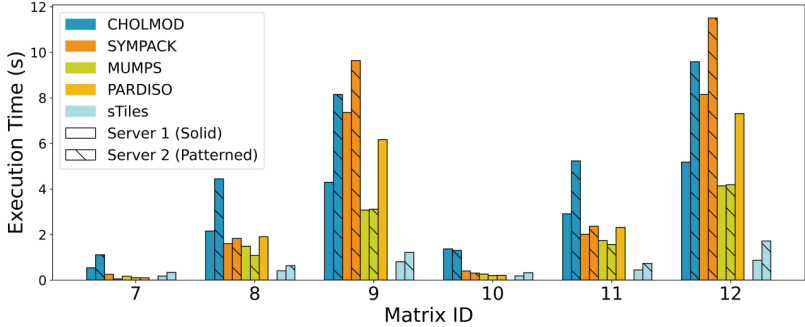
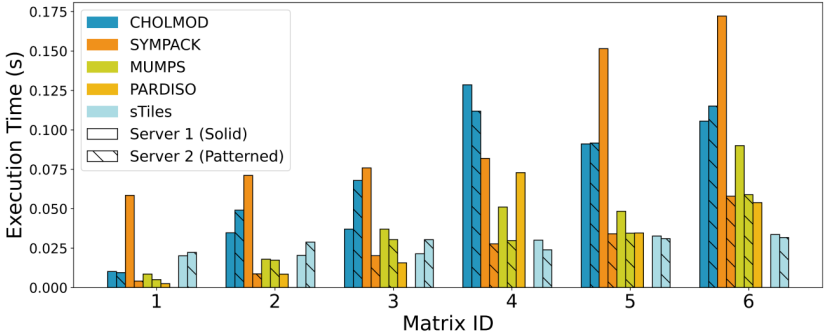
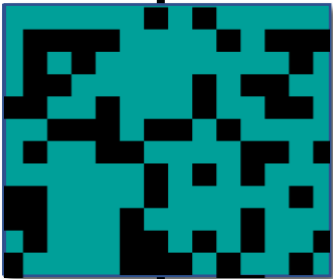
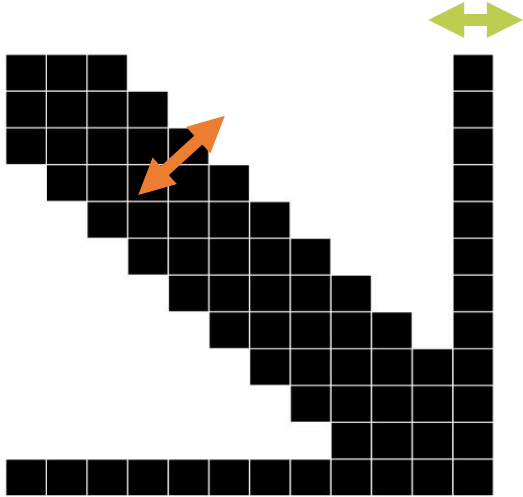


Cross-Architecture Performance Comparison



bandwidth

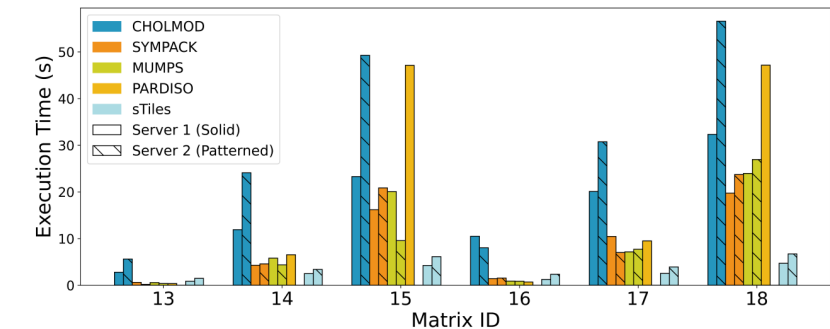
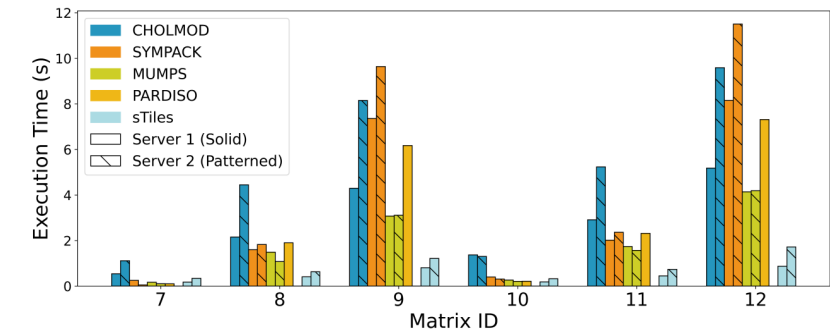
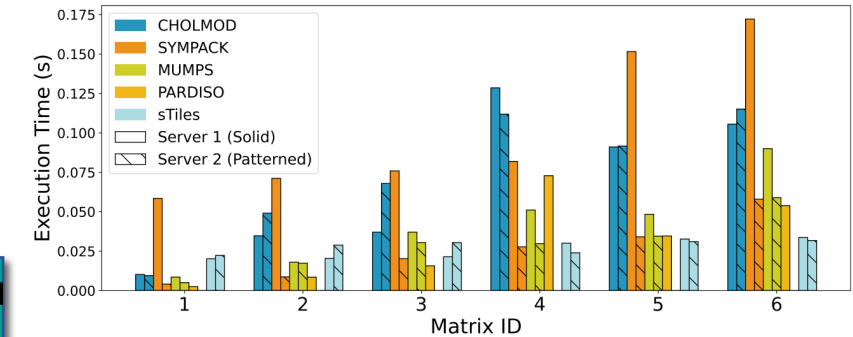
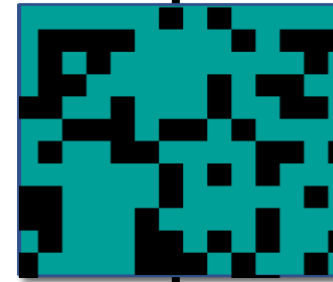
Arrowhead
thickness



Cross-Architecture Performance Comparison



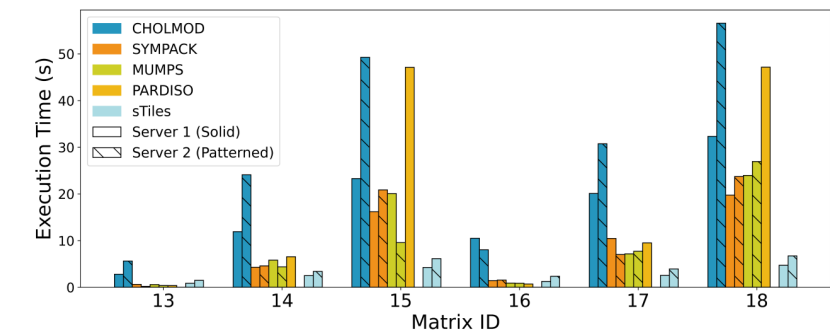
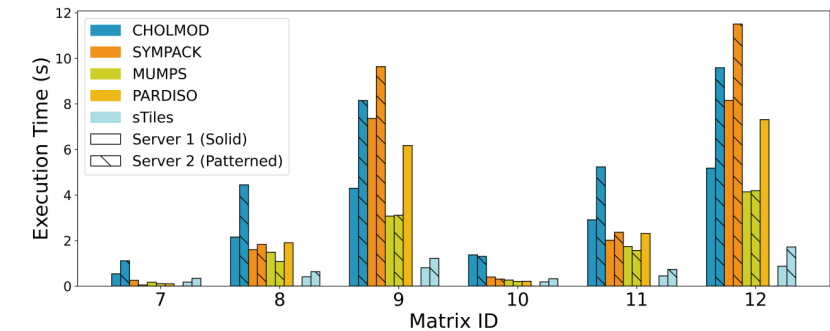
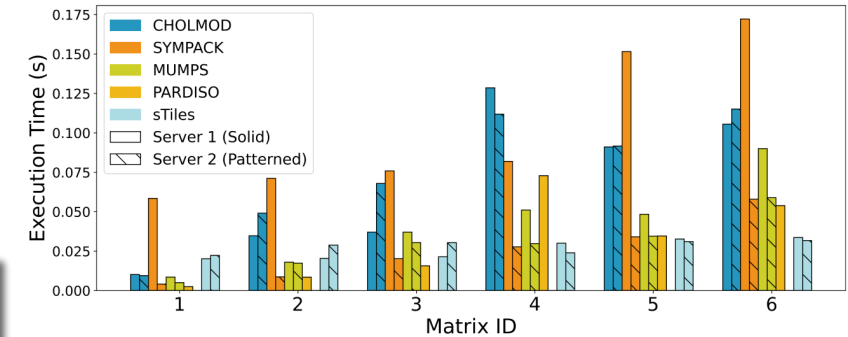
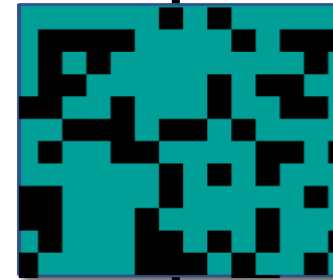
- We benchmarked sTiles against leading sparse libraries: CHOLMOD, SymPACK, MUMPS, and PARDISO.
- Experiments span various matrix sizes, bandwidths, and arrowhead structures.
 - Sizes range from 10k to 1M.
 - Arrowhead thickness: 10 and 200.
 - Densities from 0.02% to 4%.
 - Matrices mimic structures in INLA-based models.



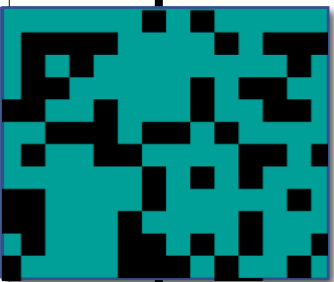
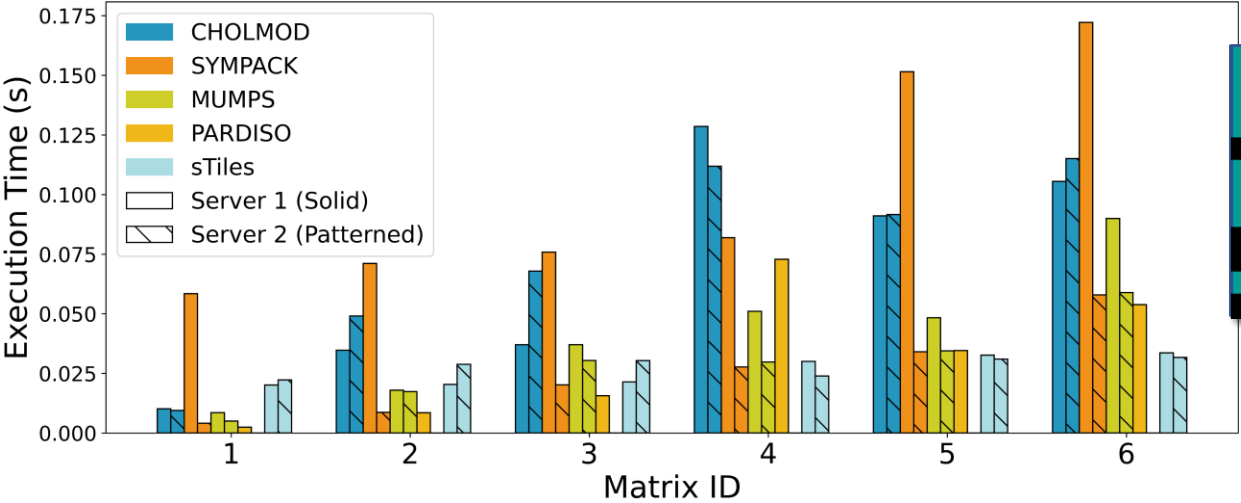
Cross-Architecture Performance Comparison



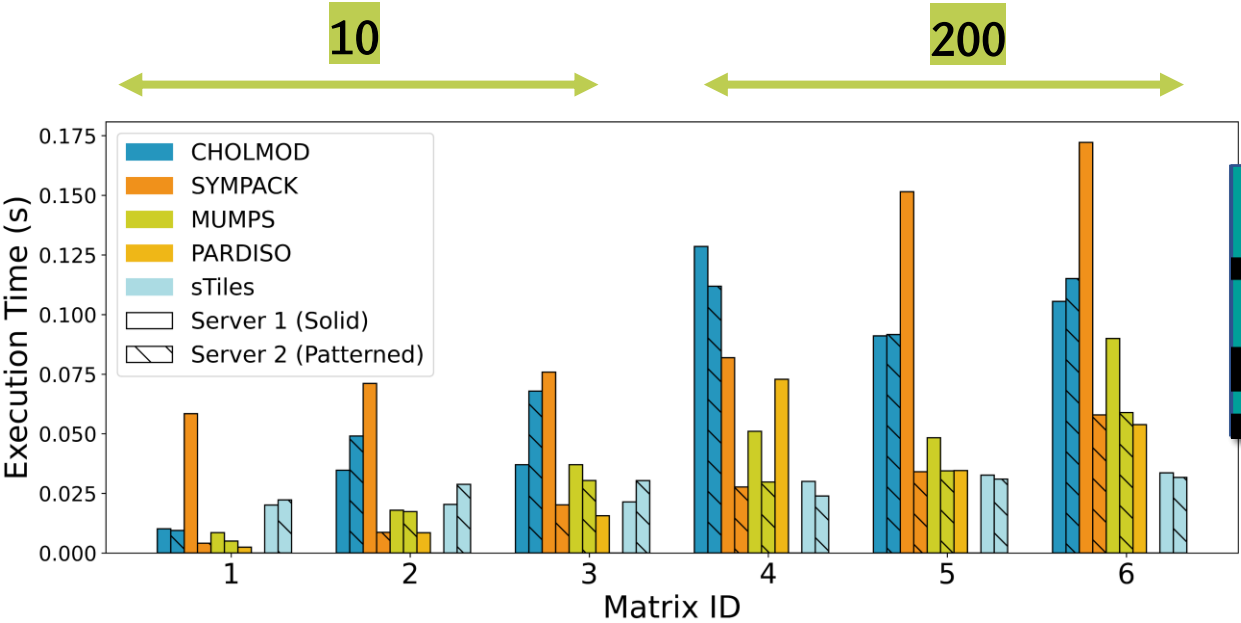
- We benchmarked sTiles against leading sparse libraries: CHOLMOD, SymPACK, MUMPS, and PARDISO.
- Experiments span various matrix sizes, bandwidths, and arrowhead structures.
 - Sizes range from 10k to 1M.
 - Arrowhead thickness: 10 and 200.
 - Densities from 0.02% to 4%.
 - Matrices mimic structures in INLA-based models.



Size: 10k

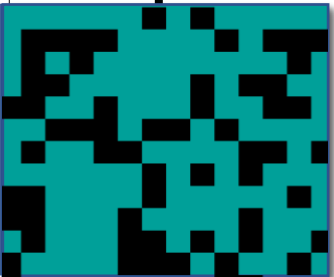
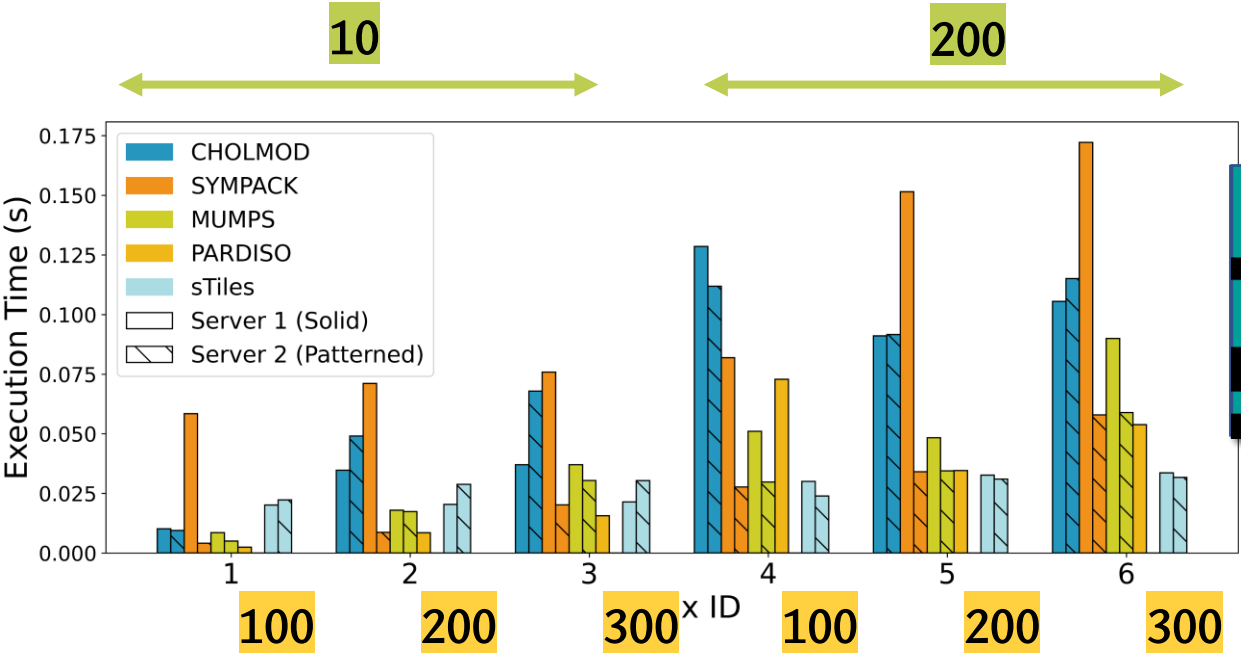


Cross-Architecture Performance Comparison



Size: 10k

Arrowhead
thickness

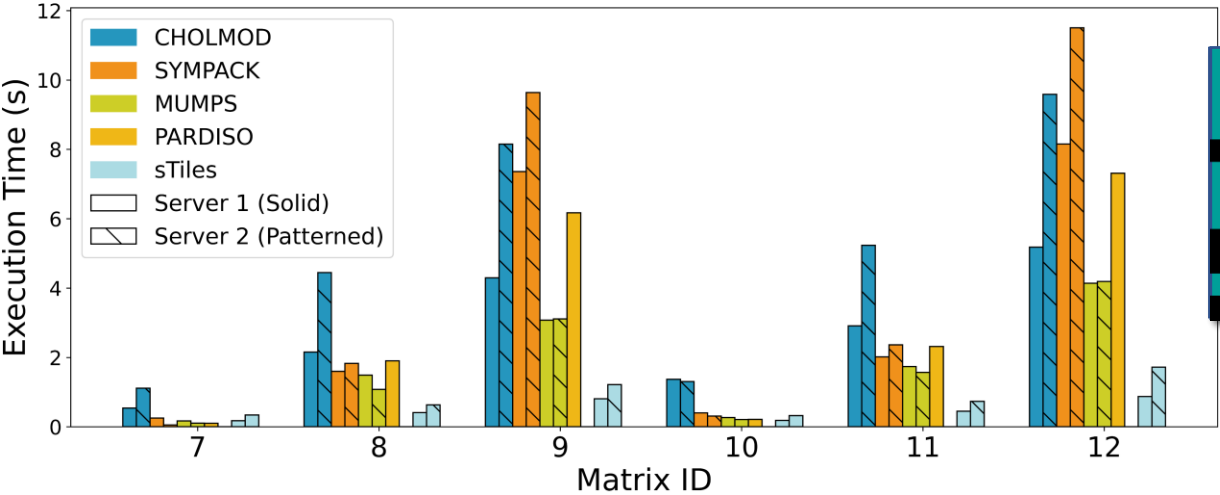


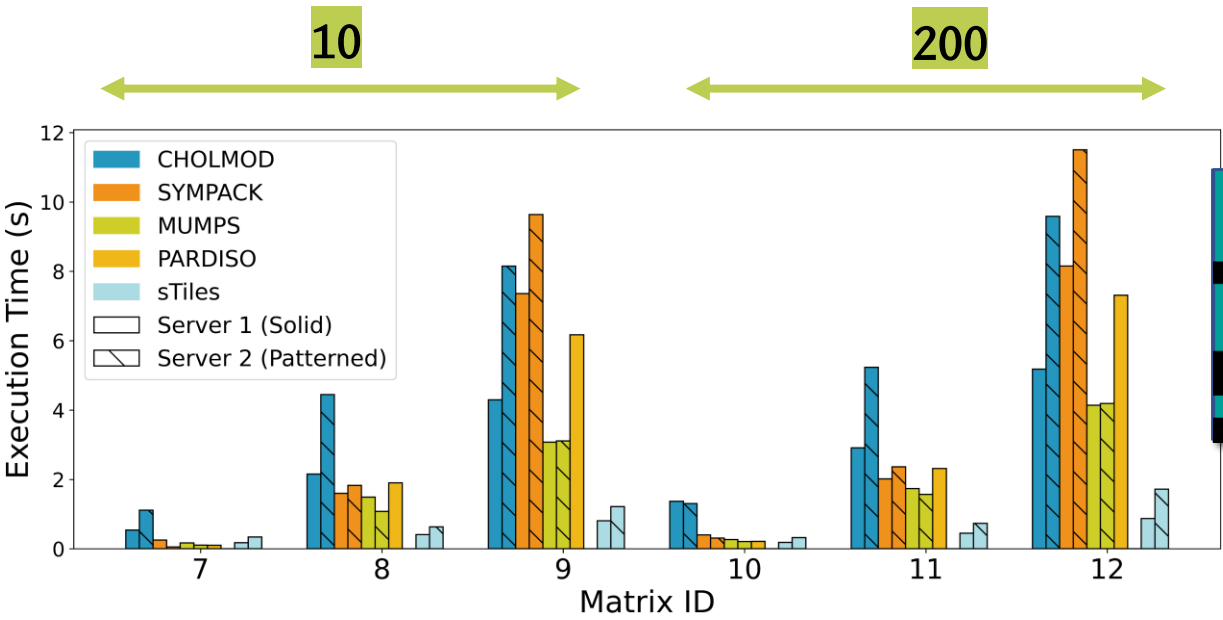
Size: 10k

Arrowhead
thickness

bandwidth

Size: 100k

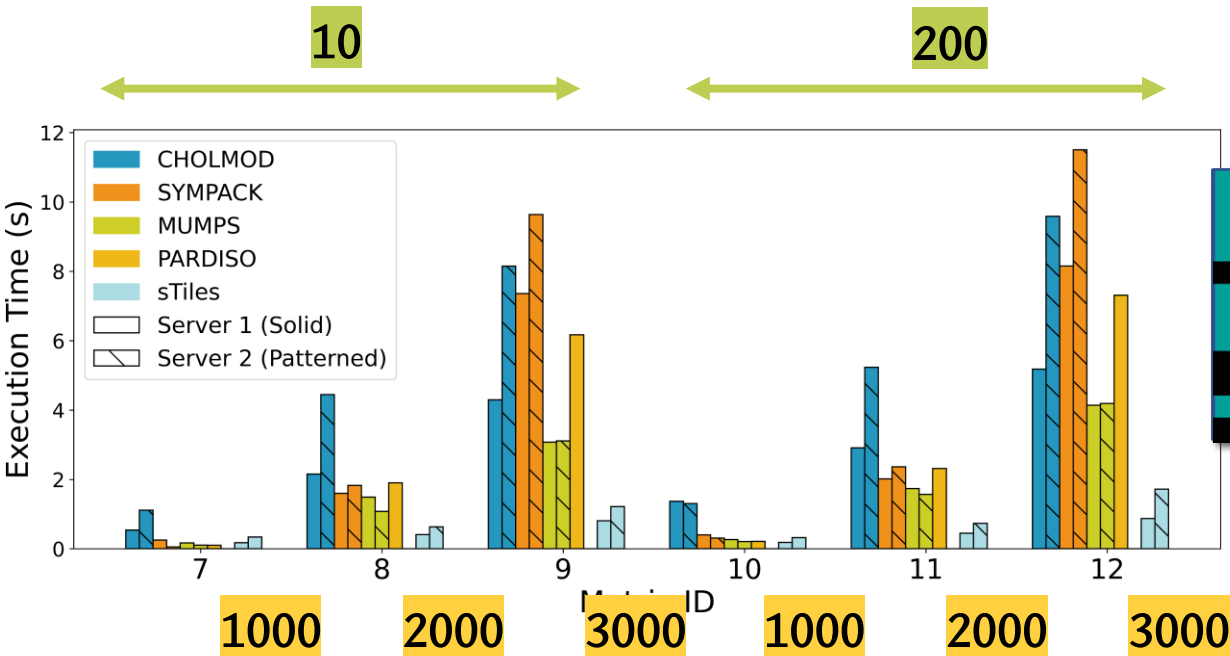


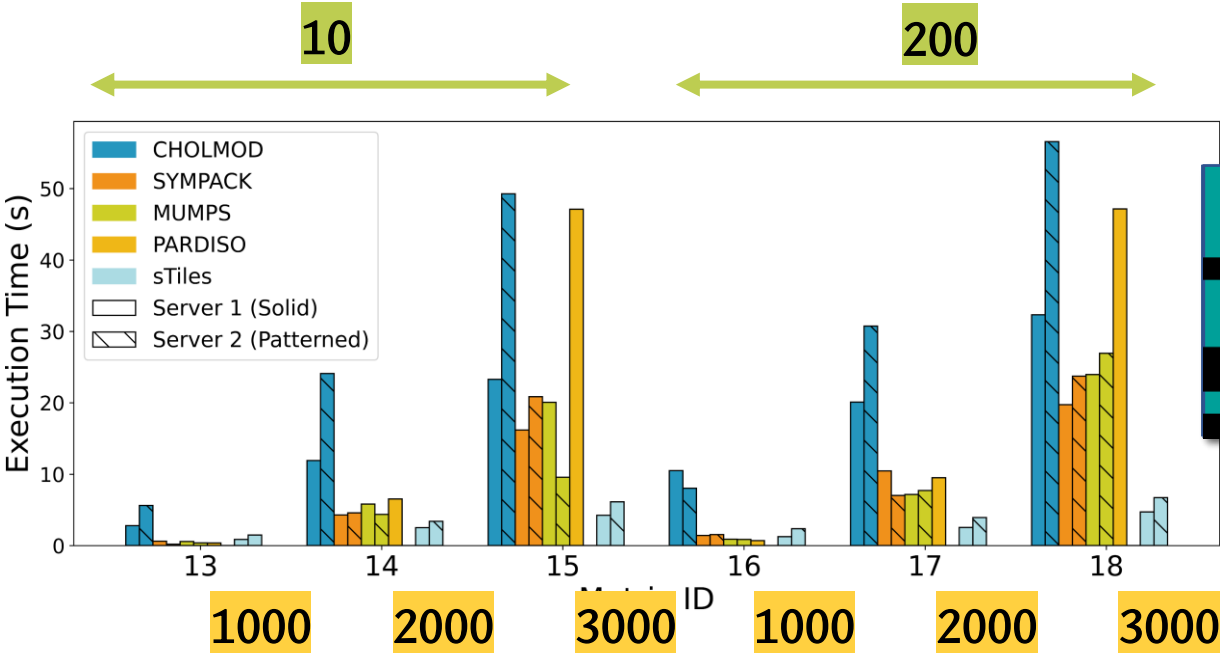


Size: 100k

Arrowhead
thickness







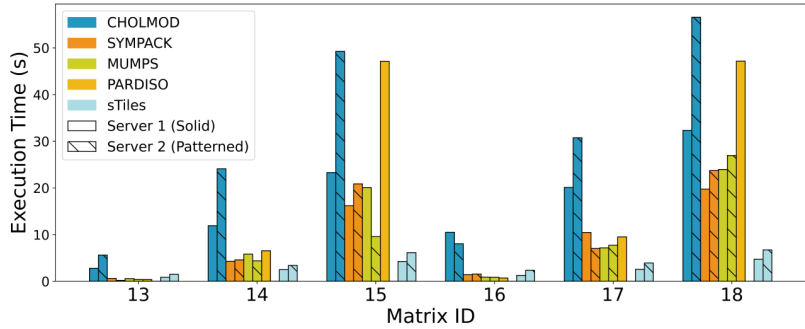
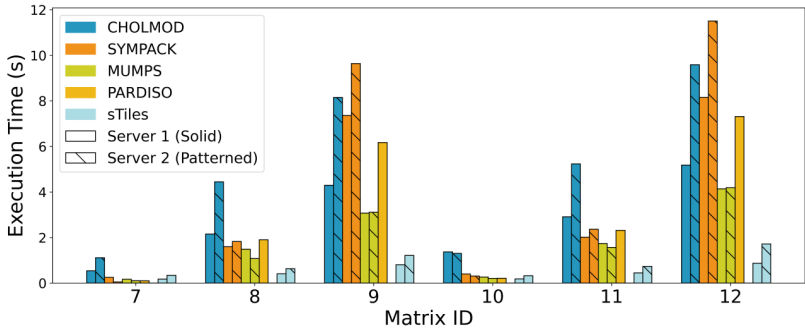
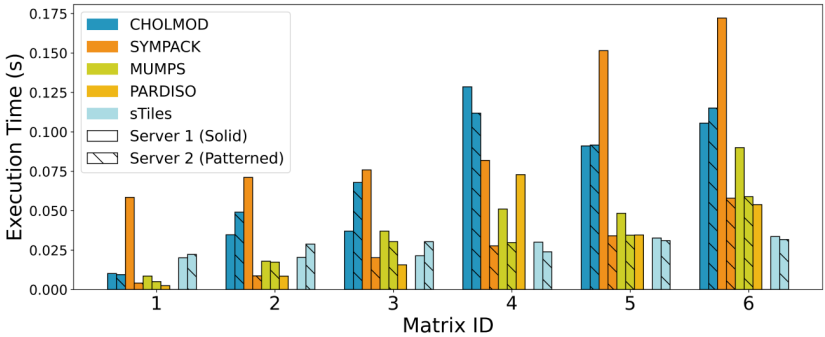
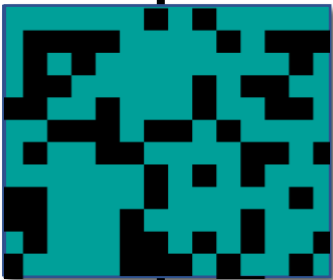
Size: 500k

Arrowhead
thickness

bandwidth

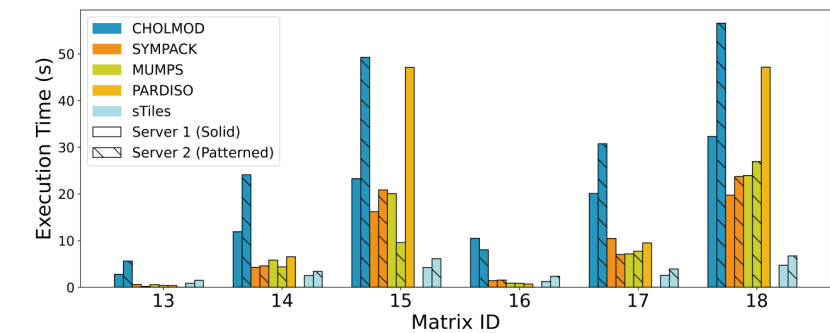
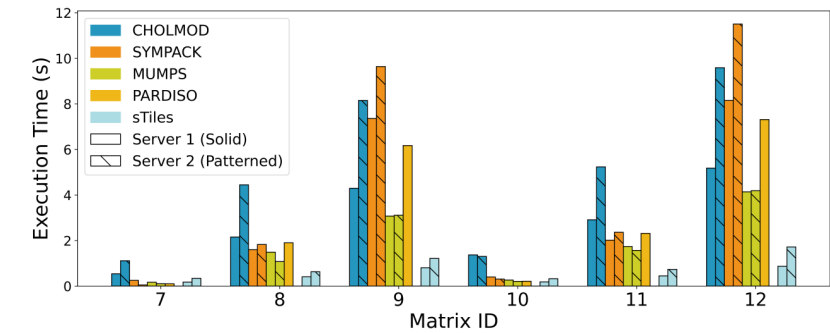
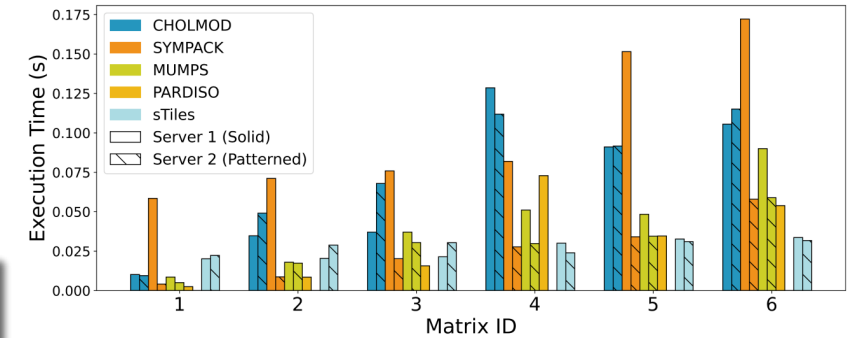
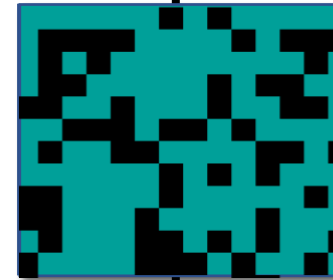


Performance Highlights:



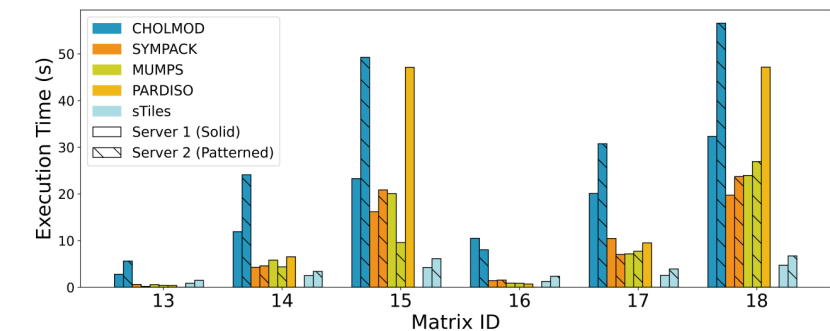
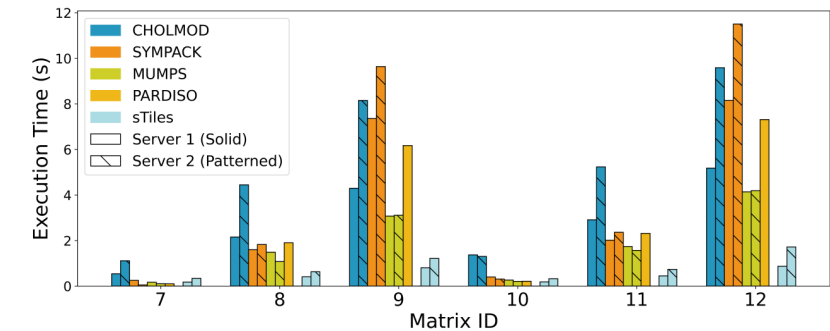
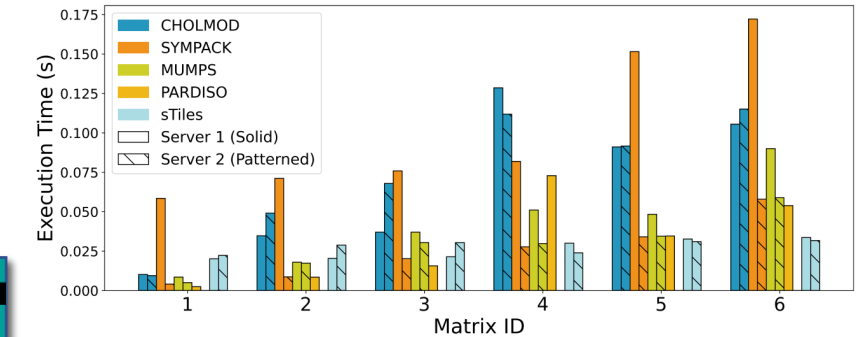
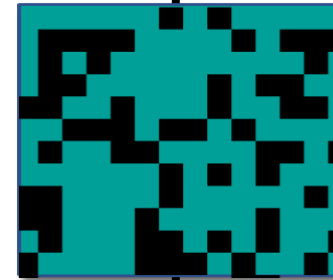
Performance Highlights:

- For small matrices (10k), all libraries perform similarly: fast and very fast.



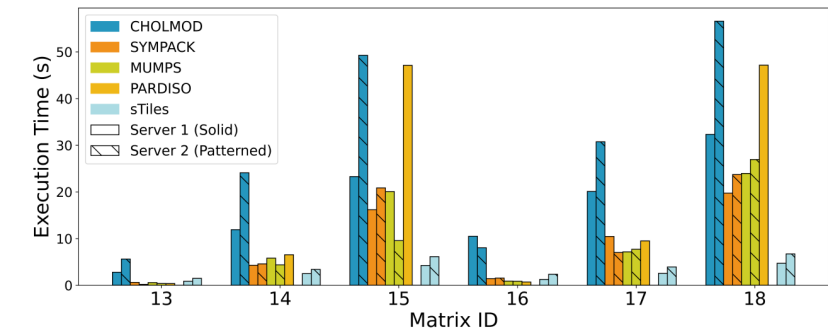
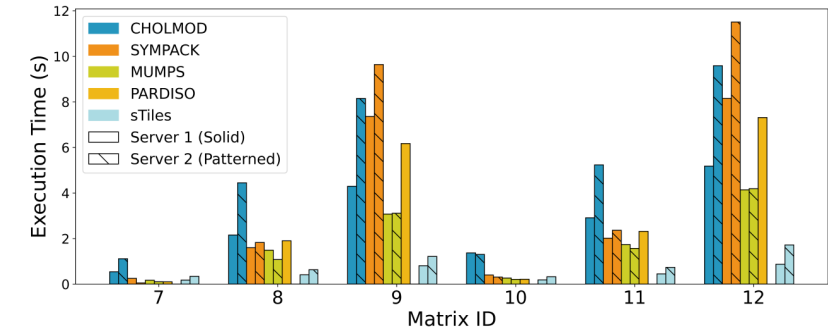
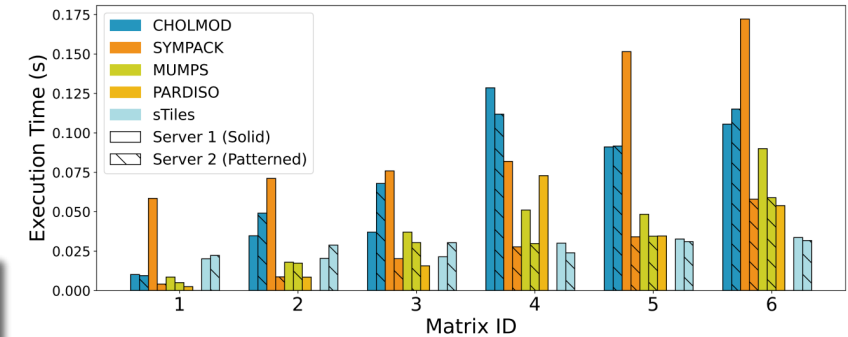
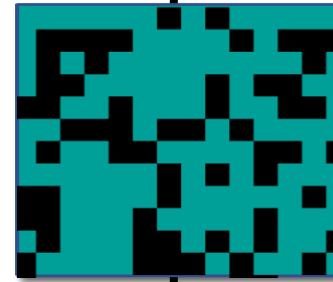
Performance Highlights:

- For small matrices (10k), all libraries perform similarly: fast and very fast.
- For 100k–500k sizes with large bandwidths and thick arrowheads, sTiles outperforms competitors.



Performance Highlights:

- For small matrices (10k), all libraries perform similarly: fast and very fast.
- For 100k–500k sizes with large bandwidths and thick arrowheads, sTiles outperforms competitors.
- Some libraries outperform sTiles in block-diagonal-only cases due to their specialized block optimizations.



Performance Highlights:

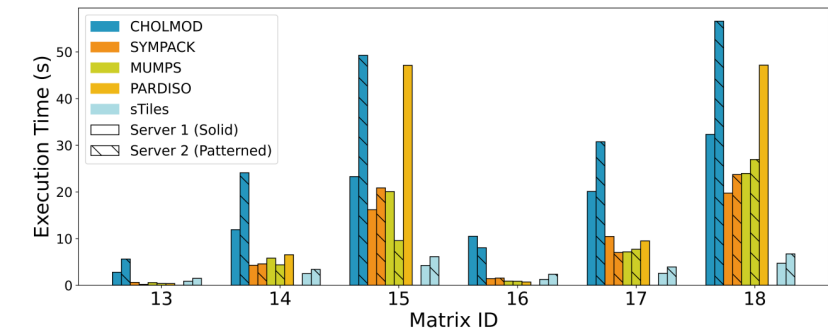
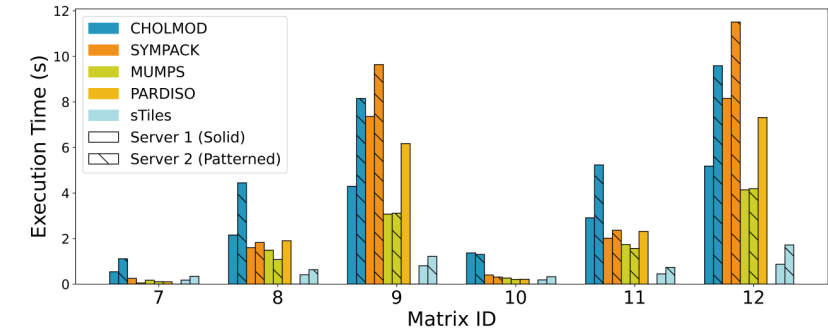
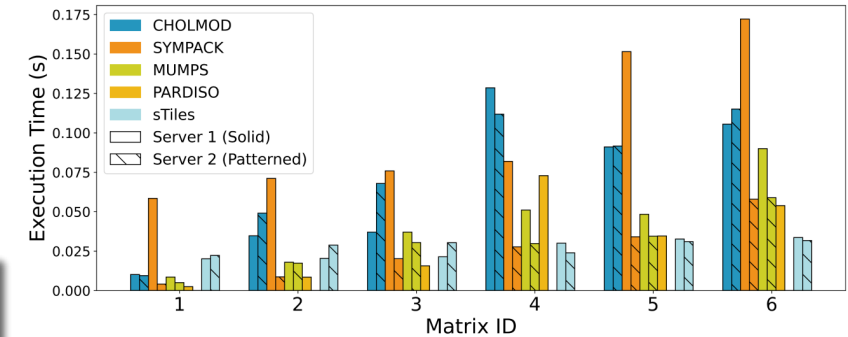
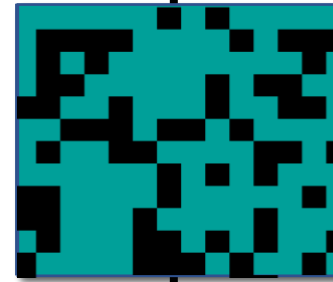
- For small matrices (10k), all libraries perform similarly: fast and very fast.
- For 100k–500k sizes with large bandwidths and thick arrowheads, sTiles outperforms competitors.
- Some libraries outperform sTiles in block-diagonal-only cases due to their specialized block optimizations.

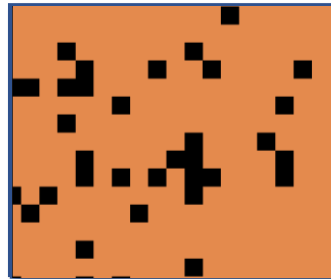
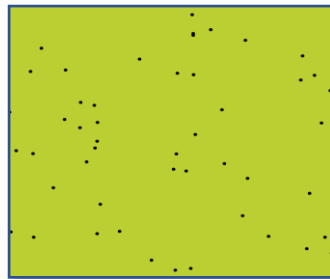
- sTiles achieving up to **8X** – CHOLMOD

9X – SymPACK

5X – MUMPS

11X – PARDISO





I. Arrowhead Sparse Matrix Factorization

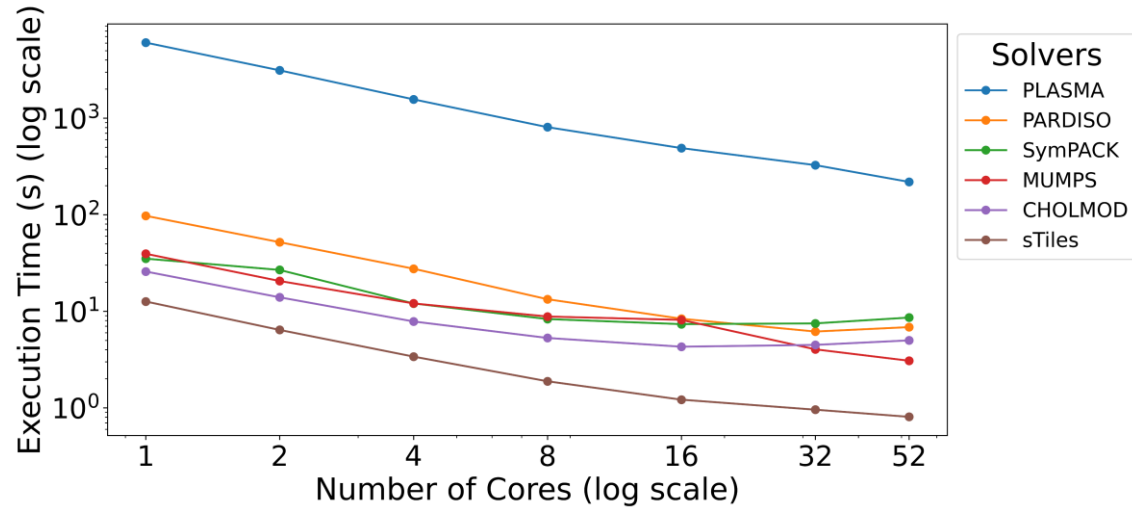
II. Parallel Optimization Techniques and GPU Acceleration

III. Performance Comparison and Experimental Results

- Cross-Architecture Performance Comparison
- Scalability Analysis
- Tree Reduction Performance
- GPU Acceleration Example

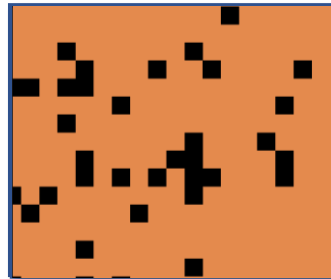
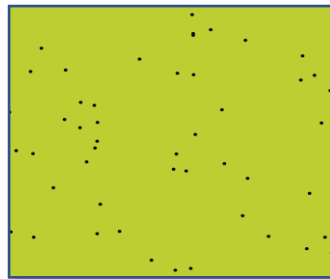
IV. Conclusion and Outlook





- sTiles achieves the best scalability across core counts, outperforming other libraries up to 52 cores, while dense solvers like PLASMA lag due to higher overhead.





I. Arrowhead Sparse Matrix Factorization

II. Parallel Optimization Techniques and GPU Acceleration

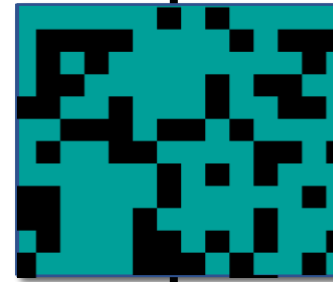
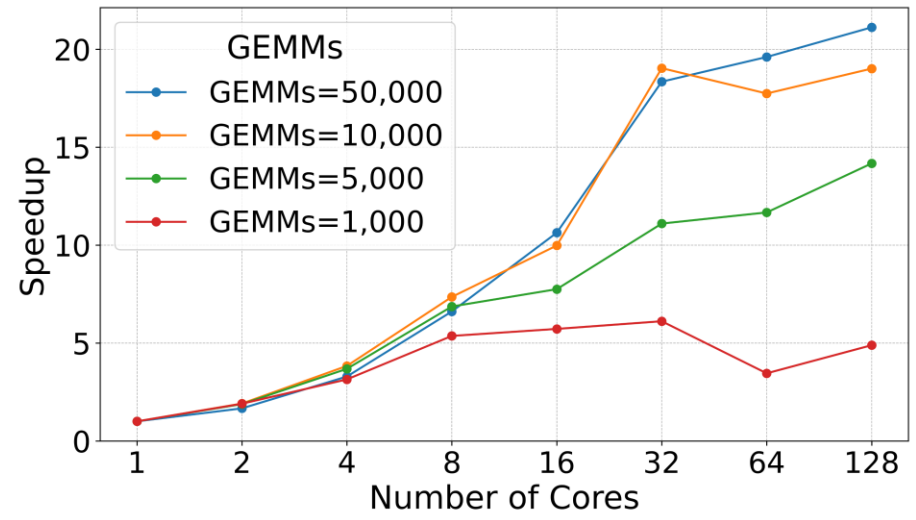
III. Performance Comparison and Experimental Results

- Cross-Architecture Performance Comparison
- Scalability Analysis
- Tree Reduction Performance
- GPU Acceleration Example

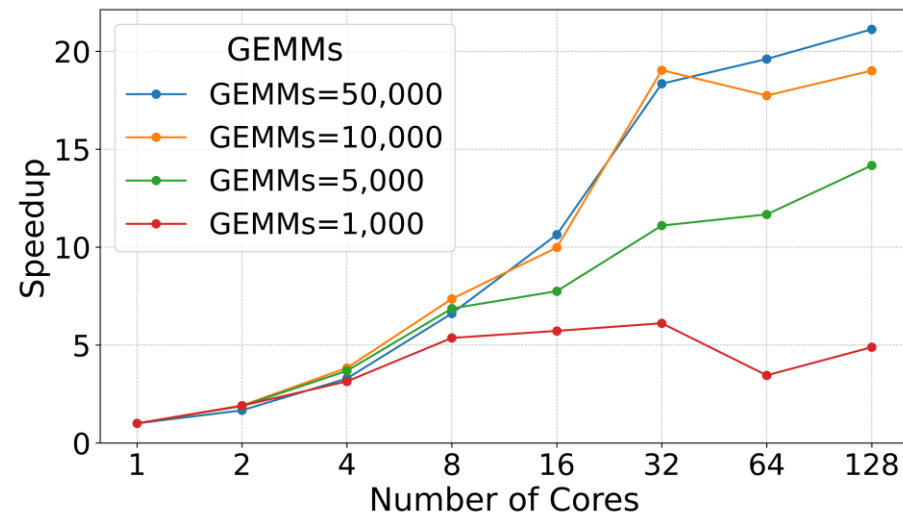
IV. Conclusion and Outlook



Experiment 1: Evaluates how tree reduction improves Cholesky factorization performance as the number of cores increases.

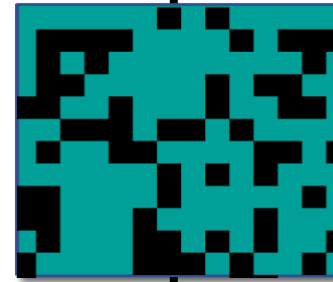
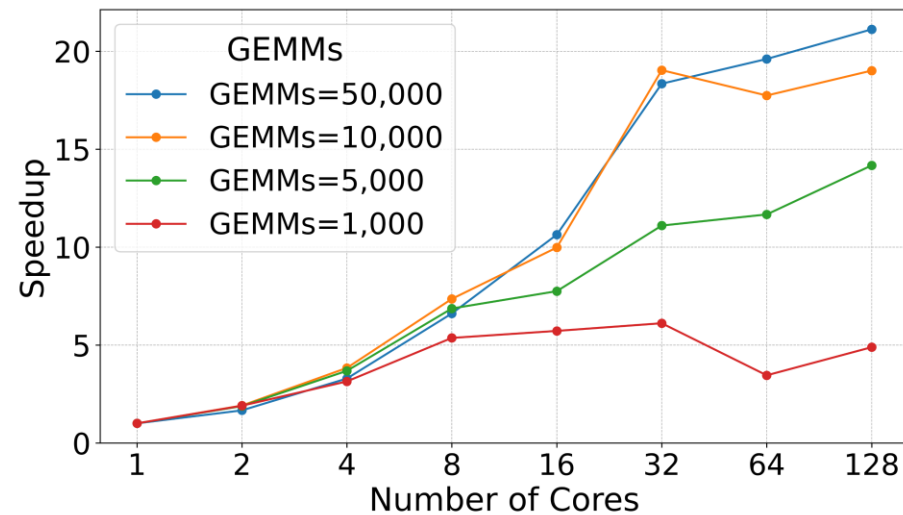


Experiment 1: Evaluates how tree reduction improves Cholesky factorization performance as the number of cores increases.



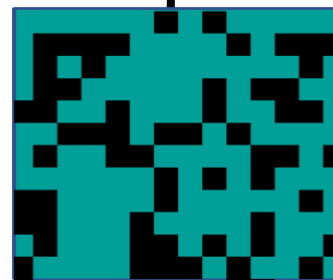
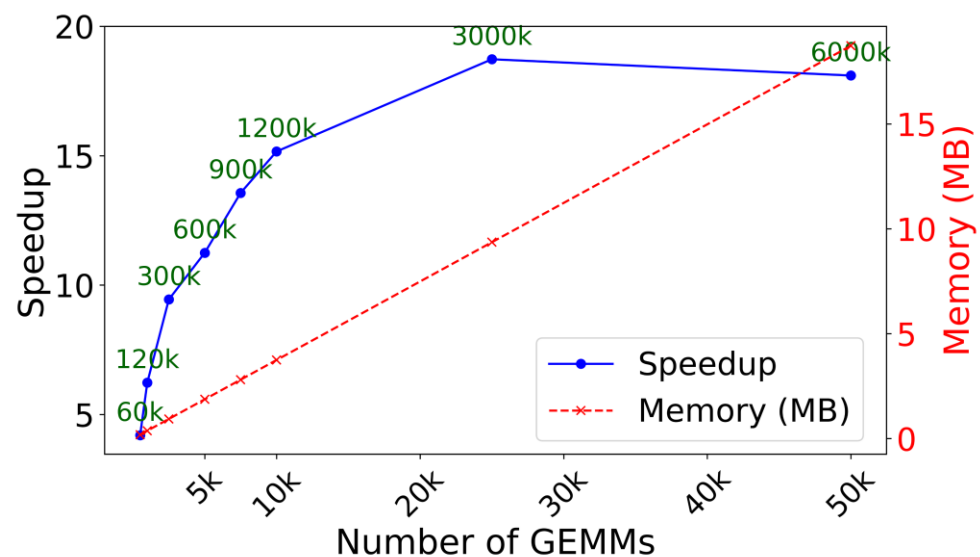
- Tree reduction achieves up to 20X speedup, with gains increasing as more cores are utilized.

Experiment 1: Evaluates how tree reduction improves Cholesky factorization performance as the number of cores increases.

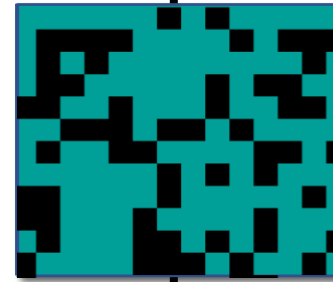
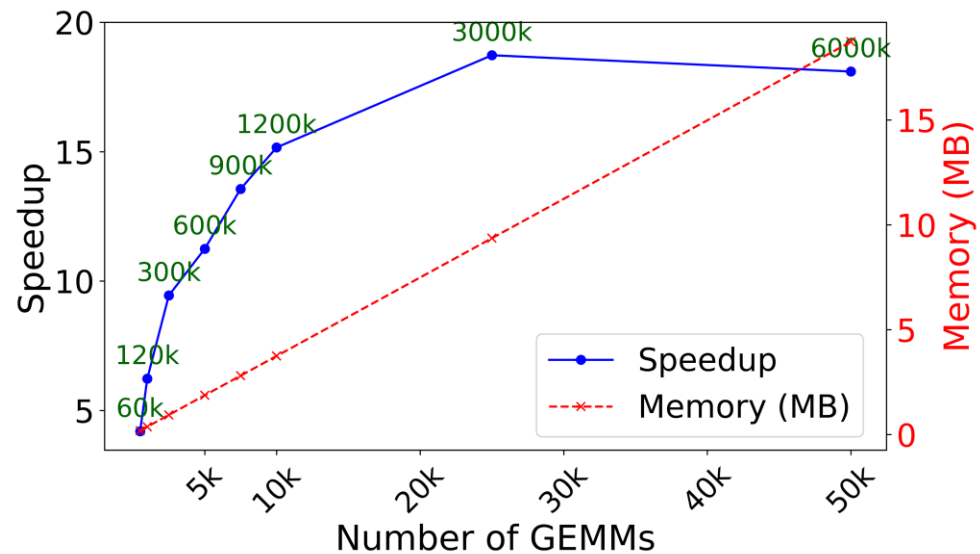


- Tree reduction achieves up to 20X speedup, with gains increasing as more cores are utilized.
- Performance scales well with core count, especially for large matrices with many GEMM/SYRK tasks.

Experiment 2: Measures the effect of matrix size on speedup and memory usage when using tree reduction with 32 cores.



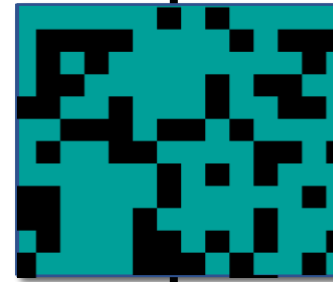
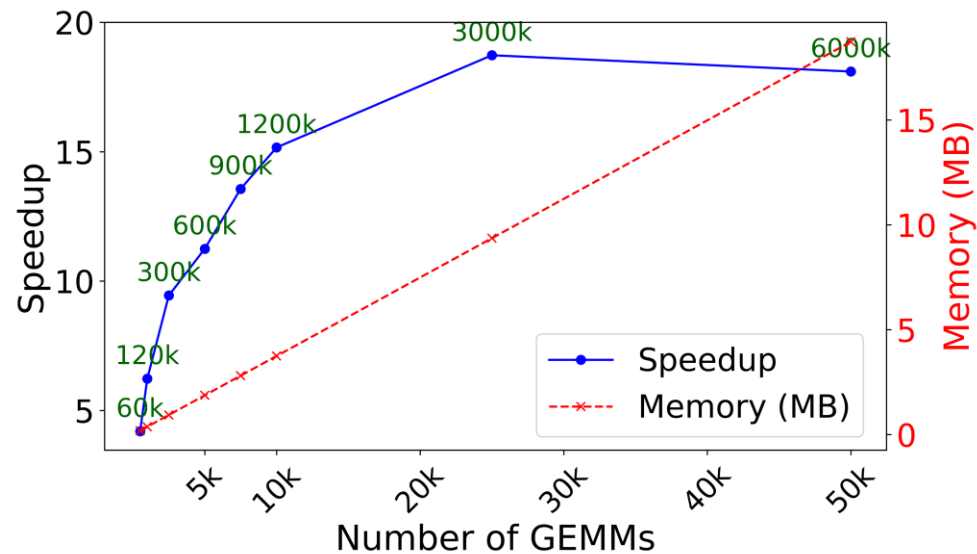
Experiment 2: Measures the effect of matrix size on speedup and memory usage when using tree reduction with 32 cores.



- Speedup improves with larger matrices, showing better compute-to-memory ratio.

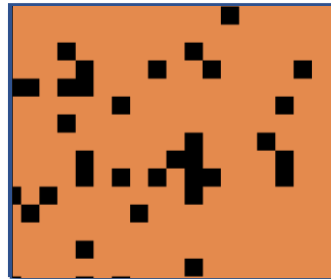
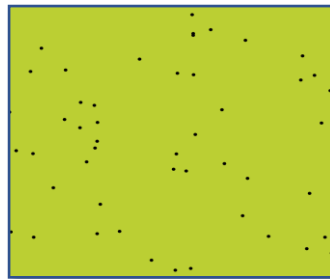


Experiment 2: Measures the effect of matrix size on speedup and memory usage when using tree reduction with 32 cores.



- Speedup improves with larger matrices, showing better compute-to-memory ratio.
- Memory usage increases moderately, but remains manageable as concurrency grows.





I. Arrowhead Sparse Matrix Factorization

II. Parallel Optimization Techniques and GPU Acceleration

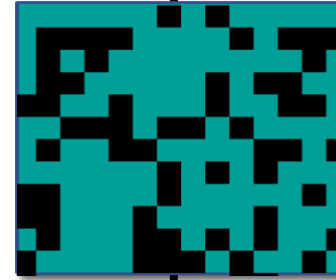
III. Performance Comparison and Experimental Results

- Cross-Architecture Performance Comparison
- Scalability Analysis
- Tree Reduction Performance
- GPU Acceleration Example

IV. Conclusion and Outlook



- We evaluate GPU performance using two large matrices to assess speedup against multi-core CPUs.

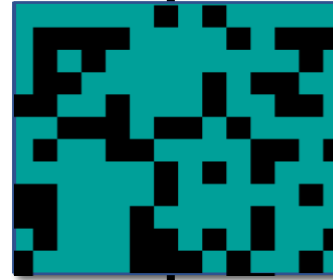


- We evaluate GPU performance using two large matrices to assess speedup against multi-core CPUs.

- Matrices:

ID 19 (size = 50,010, bandwidth = 15,000)

ID 20 (size = 1,000,010, bandwidth = 3,000)

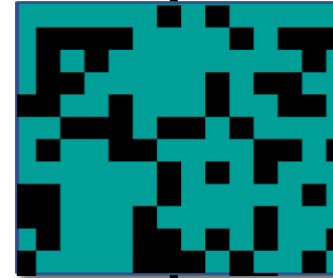


- We evaluate GPU performance using two large matrices to assess speedup against multi-core CPUs.

- Matrices:
ID 19 (size = 50,010, bandwidth = 15,000)
ID 20 (size = 1,000,010, bandwidth = 3,000)

Cores/Streams	Matrix Size: 50,010	
	Time (CPU)	Time (GPU)
1	97.66	1.46
2	48.96	1.09
4	24.91	0.94
8	12.68	0.86
16	6.53	0.79
32	4.07	0.81
64	3.68	0.96

Matrix ID 19



GPU version uses tile size = 600,
CPU version uses tile size = 120



- We evaluate GPU performance using two large matrices to assess speedup against multi-core CPUs.

- Matrices:
ID 19 (size = 50,010, bandwidth = 15,000)
ID 20 (size = 1,000,010, bandwidth = 3,000)

Cores/Streams	Matrix Size: 50,010	
	Time (CPU)	Time (GPU)
1	97.66	1.46
2	48.96	1.09
4	24.91	0.94
8	12.68	0.86
16	6.53	0.79
32	4.07	0.81
64	3.68	0.96

Matrix ID 19



GPU version uses tile size = 600,
CPU version uses tile size = 120

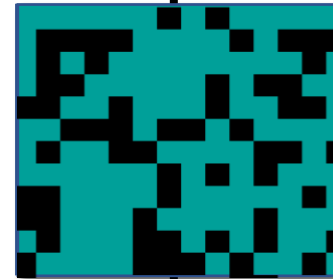


- We evaluate GPU performance using two large matrices to assess speedup against multi-core CPUs.

- Matrices:
ID 19 (size = 50,010, bandwidth = 15,000)
ID 20 (size = 1,000,010, bandwidth = 3,000)

Cores/Streams	Matrix Size: 1,000,010	
	Time (CPU)	Time (GPU)
1	239.73	9.24
2	106.18	6.61
4	60.18	5.89
8	31.05	5.75
16	20.12	5.76
32	15.95	5.75
64	24.09	9.22

Matrix ID 20



GPU version uses tile size = 600,
CPU version uses tile size = 120

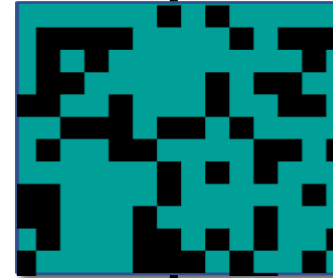


- We evaluate GPU performance using two large matrices to assess speedup against multi-core CPUs.

- Matrices:
ID 19 (size = 50,010, bandwidth = 15,000)
ID 20 (size = 1,000,010, bandwidth = 3,000)

Cores/Streams	Matrix Size: 1,000,010	
	Time (CPU)	Time (GPU)
1	239.73	9.24
2	106.18	6.61
4	60.18	5.89
8	31.05	5.75
16	20.12	5.76
32	15.95	5.75
64	24.09	9.22

Matrix ID 20



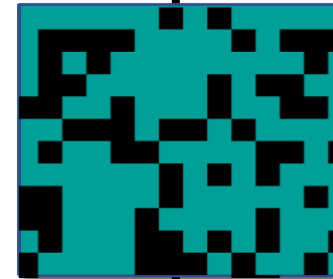
GPU version uses tile size = 600,
CPU version uses tile size = 120



- We evaluate GPU performance using two large matrices to assess speedup against multi-core CPUs.

Cores/Streams	Matrix Size: 1,000,010	
	Time (CPU)	Time (GPU)
1	239.73	9.24
2	106.18	6.61
4	60.18	5.89
8	31.05	5.75
16	20.12	5.76
32	15.95	5.75
64	24.09	9.22

Matrix ID 20



GPU version uses tile size = 600,
CPU version uses tile size = 120

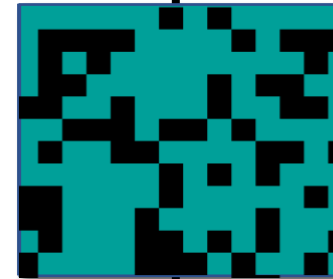
- Matrices:
ID 19 (size = 50,010, bandwidth = 15,000)
ID 20 (size = 1,000,010, bandwidth = 3,000)
- GPU implementation achieves up to
5X speedup (Matrix 19) and
2.7X speedup (Matrix 20) over 32-core CPU



- We evaluate GPU performance using two large matrices to assess speedup against multi-core CPUs.

Cores/Streams	Matrix Size: 1,000,010	
	Time (CPU)	Time (GPU)
1	239.73	9.24
2	106.18	6.61
4	60.18	5.89
8	31.05	5.75
16	20.12	5.76
32	15.95	5.75
64	24.09	9.22

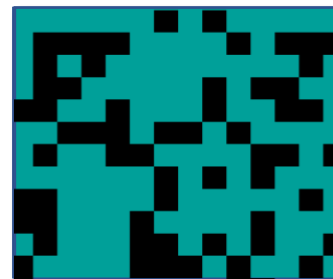
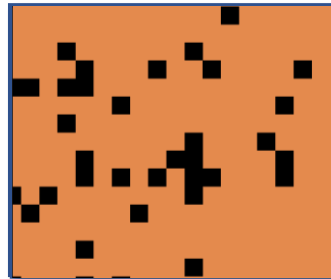
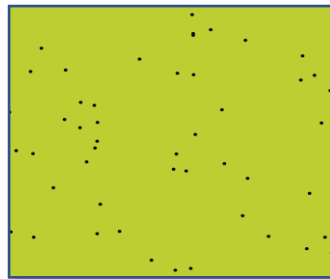
Matrix ID 20



GPU version uses tile size = 600,
CPU version uses tile size = 120

- Matrices:
ID 19 (size = 50,010, bandwidth = 15,000)
ID 20 (size = 1,000,010, bandwidth = 3,000)
- GPU implementation achieves up to
5X speedup (Matrix 19) and
2.7X speedup (Matrix 20) over 32-core CPU
- Speedup depends on **matrix size**, **bandwidth**,
and **data movement overhead**.





I. Arrowhead
Sparse Matrix
Factorization

II. Parallel
Optimization
Techniques and
GPU
Acceleration

III. Performance
Comparison &
Experimental
Results

IV. Conclusion
and Outlook

- sTiles delivers a high-performance, structure-aware framework for Cholesky factorization of sparse matrices with dense clusters.



Conclusion and
Outlook



- sTiles delivers a high-performance, structure-aware framework for Cholesky factorization of sparse matrices with dense clusters.
- It bridges the gap between sparse and dense solvers, achieving up to 11× speedup over state-of-the-art libraries.



Conclusion and
Outlook



- sTiles delivers a high-performance, structure-aware framework for Cholesky factorization of sparse matrices with dense clusters.
- It bridges the gap between sparse and dense solvers, achieving up to 11× speedup over state-of-the-art libraries.
- GPU acceleration provides significant gains for wide-bandwidth matrices.



Conclusion and
Outlook



- sTiles delivers a high-performance, structure-aware framework for Cholesky factorization of sparse matrices with dense clusters.
- It bridges the gap between sparse and dense solvers, achieving up to 11× speedup over state-of-the-art libraries.
- GPU acceleration provides significant gains for wide-bandwidth matrices.
- Designed to scale for structured precision matrices in applications like INLA and beyond.



Conclusion and
Outlook

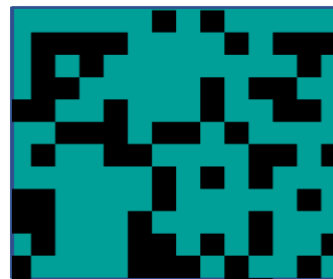
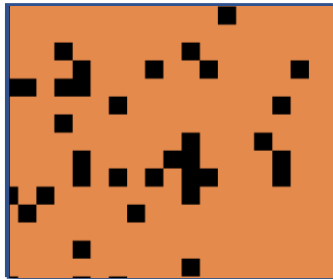
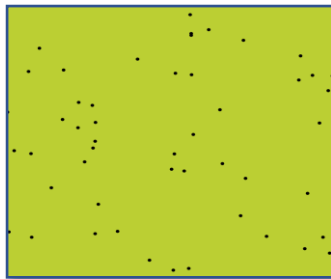


- sTiles delivers a high-performance, structure-aware framework for Cholesky factorization of sparse matrices with dense clusters.
- It bridges the gap between sparse and dense solvers, achieving up to $11\times$ speedup over state-of-the-art libraries.
- GPU acceleration provides significant gains for wide-bandwidth matrices.
- Designed to scale for structured precision matrices in applications like INLA and beyond.
- A new extension targets selected elements of a sparse matrix via a parallel two-phase tile algorithm. Up to $13\times$ faster than Panua-PARDISO on CPU, and $5\times$ GPU speedup on large structured matrices.



Conclusion and
Outlook





Thank you!



Thank you!

